Packet Operator Algorithms, Exact Compilation, and GPU Realization

By Komorebi P.

Abstract

A packet operator is a linear map written between the members of a finite direct sum. We study the resulting computational representation before choosing a processor or kernel. Exact formulas are given for block sparse application, block diagonal plus low rank algebra, Woodbury inversion, support propagation, block preconditioning, and packetwise perturbation. Products of low rank corrections admit optimal recompression through a small singular value problem, with explicit error propagation through a product chain. A rooted candidate decomposition supports an exact compiler when representation cost and error are additive. Dynamic programming gives the optimal Lagrangian pruning, an exact quantized error budget algorithm, and a local update rule. Discrete precision assignment becomes a multiple choice budget problem. Arithmetic, storage, data movement, launch count, discovery, transfer, and materialization are kept as separate costs. A sanitized corpus of 154 recorded CPU and GPU measurements illustrates regular packet gains, precision frontiers, crossover behavior, and retained negative controls. The largest matched structured result is a factor of 14.89 against a device resident dense GPU calculation. The paper proves finite algebraic and optimization statements. Hardware measurements remain specific to their recorded environment, and no generic dense matrix exponent or universal discovery algorithm is claimed.

1. Introduction

A matrix can look dense while its action is not. Independent diagonal blocks, sparse transfers, low rank couplings, tensor factors, and transform coordinates can remove most of the arithmetic and storage before a kernel is launched. The gain is mathematical first. A device can exploit only the structure that the representation exposes.

Keywords: structured matrix, low rank update, dynamic programming, mixed precision, GPU computation, benchmark contract

AMS Classification: Primary: 65F30, Secondary: 65F50, 65Y05, 68W10, 68W40.

This article begins with a finite direct sum

$$X = \bigoplus_{\alpha \in I} X_\alpha$$

and a complete family of coordinate projectors

$$P_\alpha P_\beta = \delta_{\alpha\beta} P_\alpha, \qquad \sum_{\alpha \in I} P_\alpha = I.$$

An operator has packet blocks

$$A_{\beta\alpha} = P_\beta A P_\alpha.$$

The active addresses form a directed graph. Block dimensions, block types, approximation tolerances, and output requirements turn this graph into a computational intermediate representation.

The underlying linear algebra is classical. Direct and iterative methods are treated in [14], [24], and [9]. Low rank approximation and updates are governed by singular value decomposition and the Sherman Morrison Woodbury identity [11], [21], [15], and [4]. Floating point analysis follows the standard model in [17] and [19]. Randomized discovery of low rank structure is developed in [16] and [20]. Communication and attainable performance are shaped by data movement and arithmetic intensity [18], [26], [2], and [12].

The first task is semantic. A compressed representation must specify the operator that it denotes. It must also specify what an algorithm returns. A product represented by block and low rank factors is not the same output as an explicit matrix. A timing comparison is valid only after the output contract and timing scope agree.

The second task is algebraic. Block diagonal plus low rank operators are closed under multiplication before recompression. Their unrecompressed rank grows additively. Woodbury inversion reduces a large inverse to packet solves and one small dense inverse. The low rank factors can then be recompressed by orthogonalizing them and taking the singular value decomposition of a small core. This produces the best truncated approximation in spectral and Frobenius norm.

The third task is combinatorial. Sparse transfer graphs predict possible support under powers, polynomials, and Schur elimination. A directed acyclic graph gives a finite path expansion. A block triangular solve can be scheduled by topological level. Graph regularity matters because equal shapes can be batched, while irregular edges require indexing and accumulation.

The fourth task is numerical. Packet perturbations can be certified by their block sensitivities. A continuous model assigns tolerances through a convex calculation. Upper bounds on available tolerances produce a clipped allocation.

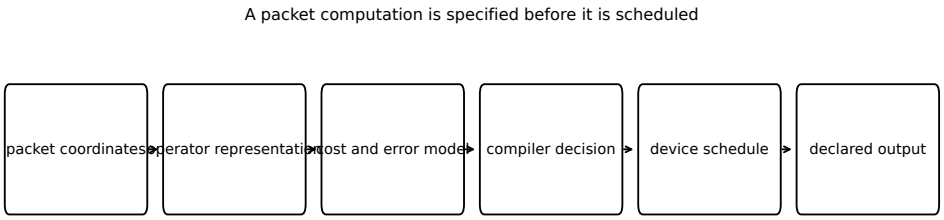


Figure 1. The representation and the output contract are fixed before the device schedule is chosen.

A finite set of data types gives a multiple choice budget problem that can be solved exactly after the errors are quantized.

The central computational result concerns compilation. Suppose a finite rooted tree records candidate refinements. Each node may be kept as one representation or split into its children. If cost and error are additive across the selected leaves, the optimal pruning is found exactly by dynamic programming. A quantized error budget has a second exact dynamic program. The decision is stable when the score gap exceeds the perturbation in the local models. A change inside one subtree requires recomputation only along the ancestor chain. These results prove a restricted packet compiler. They do not solve unrestricted discovery or arbitrary device scheduling.

The measured part uses a sanitized corpus collected on one recorded CPU and one NVIDIA GeForce RTX 3070 Ti Laptop GPU. The corpus contains 154 measured rows from sixteen suites. Regular equal block operators and block diagonal plus low rank operators show large gains. Recursive HODLR, Strassen multiplication, iterative refinement, a small Walsh Hadamard transform, and an attempted two stream overlap provide negative controls. These rows are retained because structure does not guarantee speed.

The article separates four sources of gain:

$$\text{representation}, \qquad \text{device}, \qquad \text{kernel}, \qquad \text{amortization}.$$

A single speedup number can mix all four. The benchmark protocol therefore records case identity, output contract, timing scope, source precision, correctness reference, discovery time, compilation time, and reuse count.

The mathematical statements are finite. The empirical statements apply only to the recorded environment and generated cases. The article does not claim a new generic matrix multiplication exponent. Fast matrix multiplication and its stability form a separate theory [25] and [10]. It also does not claim that every structured matrix has a profitable GPU realization.

2. **Packet operator representations**

2.1. *Exact semantics.*

Definition 2.1 (Packet layout). A packet layout is a finite family of finite dimensional Hilbert spaces

$$\mathcal{P} = (X_\alpha)_{\alpha \in I}$$

with direct sum

$$X = \bigoplus_{\alpha \in I} X_\alpha.$$

Write

$$n_\alpha = \dim X_\alpha, \qquad n = \sum_{\alpha \in I} n_\alpha.$$

The coordinate projector onto X_α is denoted by P_α.

Definition 2.2 (Packet operator representation). A finite packet operator representation consists of

$$\mathsf{IR} = (\mathcal{P}, E, (B_{\beta\alpha})_{(\beta,\alpha)\in E}),$$

where

$$E \subseteq I \times I$$

and

$$B_{\beta\alpha} : X_\alpha \longrightarrow X_\beta.$$

Its evaluation is

$$\operatorname{eval}(\mathsf{IR}) = \sum_{(\beta,\alpha)\in E} \iota_\beta B_{\beta\alpha} P_\alpha,$$

where $\iota_\beta : X_\beta \to X$ is the coordinate inclusion.

Theorem 2.3 (Exact evaluation). *Let $A \in \operatorname{End}(X)$. Set*

$$B_{\beta\alpha} = P_\beta A P_\alpha$$

and let E contain exactly the nonzero blocks. Then

$$\boxed{A = \sum_{(\beta,\alpha)\in E} P_\beta A P_\alpha.}$$

For every matrix of right hand sides Z,

$$AZ = \sum_{(\beta,\alpha)\in E} B_{\beta\alpha} P_\alpha Z.$$

Proof. Insert the identity on both sides of A:

$$A = \left(\sum_\beta P_\beta \right) A \left(\sum_\alpha P_\alpha \right) = \sum_{\alpha,\beta} P_\beta A P_\alpha.$$

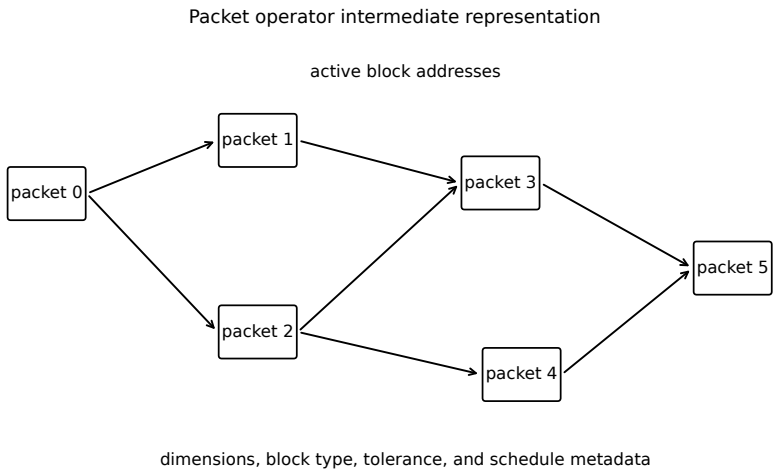


Figure 2. A packet intermediate representation stores active block addresses together with dimensions, block types, tolerances, and scheduling metadata.

The zero blocks can be removed. Applying the identity to Z gives the second formula. □

Definition 2.4 (Support graph). The support graph G_A has vertex set I. It has a directed edge

$$\alpha \longrightarrow \beta$$

when $P_\beta A P_\alpha \neq 0$.

THEOREM 2.5 (Block composition). *Let* $A, B \in \mathrm{End}(X)$. *Then*

$$\boxed{(AB)_{\gamma\alpha} = \sum_{\beta \in I} A_{\gamma\beta} B_{\beta\alpha}.}$$

Consequently,

$$\mathrm{supp}(AB) \subseteq \mathrm{supp}(A) \circ \mathrm{supp}(B),$$

where relational composition is defined by

$$(\gamma, \alpha) \in E \circ F$$

when some β *satisfies*

$$(\gamma, \beta) \in E, \qquad (\beta, \alpha) \in F.$$

Proof. Insert the packet identity between A and B:

$$P_\gamma A B P_\alpha = \sum_\beta P_\gamma A P_\beta B P_\alpha.$$

A summand can be nonzero only when both block addresses are active. □

COROLLARY 2.6 (Adjoint and tensor semantics). *The adjoint representation has blocks*

$$(A^*)_{\alpha\beta} = A^*_{\beta\alpha}.$$

If A and B are packet resolved on X and Y, then

$$(A \otimes B)_{(\beta,\delta),(\alpha,\gamma)} = A_{\beta\alpha} \otimes B_{\delta\gamma}.$$

Proof. Both identities follow by inserting the corresponding coordinate projectors and using

$$(P_\beta A P_\alpha)^* = P_\alpha A^* P_\beta.$$

The tensor identity follows from

$$(P_\beta \otimes Q_\delta)(A \otimes B)(P_\alpha \otimes Q_\gamma) = A_{\beta\alpha} \otimes B_{\delta\gamma}.$$

□

2.2. *Output contracts.*

Definition 2.7 (Output contract). An output contract specifies the mathematical object returned by an algorithm. The contract may require an explicit vector, an explicit matrix, block factors, low rank factors, spectral factors, or an implicit operator with a declared application rule.

Definition 2.8 (Timing scope). A timing scope states which costs are included. Possible costs include structure discovery, compilation, host transfer, device transfer, kernel execution, graph capture, graph replay, factor reconstruction, and explicit materialization.

PROPOSITION 2.9 (Contract conversion). *Suppose one algorithm returns a compressed representation R and another returns an explicit object Y. A direct timing comparison is valid only if*

$$\mathrm{decode}(R) = Y$$

and the time required by decode *is included, or if the downstream task consumes R directly.*

Proof. Without decoding, the two procedures return different mathematical objects. A timing ratio between different outputs does not compare implementations of the same map. Including the decoding cost gives two procedures with the same output. If the next operation consumes R, decoding is not part of the declared computation. □

Remark 2.10. A compressed product and an explicit matrix product may both be exact. They still have different contracts. This distinction becomes decisive when materialization costs n^2 entries.

3. **Arithmetic, storage, and data movement**

We count one scalar multiplication and one scalar addition as two modeled floating point operations. These counts describe a representation. They do not predict wall time without a memory and launch model.

THEOREM 3.1 (Dense and packet application counts). *A dense n by n matrix applied to q right hand sides has modeled cost*

$$\boxed{\mathrm{flop}_{\mathrm{dense}} = 2n^2 q.}$$

A packet representation with active edge set E has modeled cost

$$\boxed{\mathrm{flop}_{\mathrm{packet}} = 2q \sum_{(\beta,\alpha)\in E} n_\beta n_\alpha.}$$

It stores

$$\boxed{M_{\mathrm{packet}} = \sum_{(\beta,\alpha)\in E} n_\beta n_\alpha}$$

scalar entries.

Proof. The active block $B_{\beta\alpha}$ is an n_β by n_α matrix. Applying it to an n_α by q input costs $2n_\beta n_\alpha q$ modeled operations and stores $n_\beta n_\alpha$ scalars. Summation over the active addresses gives both formulas. □

COROLLARY 3.2 (Equal diagonal packets). *Suppose n is divided into equal diagonal packets of width b. Then*

$$\mathrm{flop}_{\mathrm{block}} = 2qnb, \qquad M_{\mathrm{block}} = nb.$$

The dense to block reduction factor is

$$\boxed{\frac{n}{b}.}$$

Proof. There are n/b active diagonal blocks. Each contains b^2 entries. □

Definition 3.3 (Block diagonal plus low rank operator). A block diagonal plus low rank operator has the form

$$A = D + UV^*,$$

where

$$D = \bigoplus_{\alpha\in I} D_\alpha, \qquad U, V \in \mathbb{C}^{n\times r}.$$

The abbreviation BDLR will be used below.

THEOREM 3.4 (BDLR application and storage). *For q right hand sides,*

$$AZ = DZ + U(V^*Z)$$

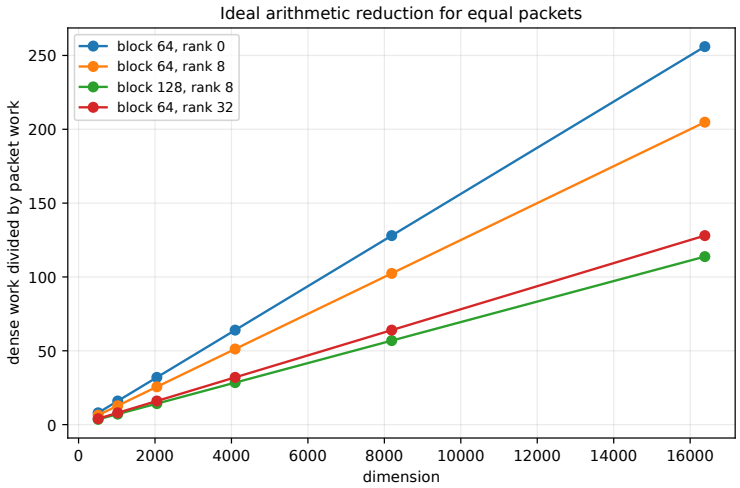


Figure 3. The ideal reduction grows with dimension only when packet width and transfer rank remain controlled.

has modeled cost

$$\boxed{2q\left(\sum_{\alpha} n_{\alpha}^{2} + 2nr\right).}$$

The representation stores

$$\boxed{\sum_{\alpha} n_{\alpha}^{2} + 2nr}$$

scalars. For equal packets of width b*, both ideal dense reduction factors equal*

$$\boxed{\frac{n}{b+2r}.}$$

Proof. The block diagonal product costs

$$2q\sum_{\alpha} n_{\alpha}^{2}.$$

The products $V^{*}Z$ and $U(V^{*}Z)$ cost $2nrq$ each. Storage consists of the diagonal blocks and the two low rank factors. For equal packets,

$$\sum_{\alpha} n_{\alpha}^{2} = nb.$$

□

Definition 3.5 (One pass streaming model). Fix a scalar size s bytes. For every active block, the one pass streaming model reads the block once, reads its source packet once, and reads and writes its target packet once. It does not assume cache reuse across edges.

PROPOSITION 3.6 (Streaming byte count). *Under definition* 3.5, *applying a packet operator to q right hand sides moves*

$$\boxed{B_{\text{stream}} = s \sum_{(\beta,\alpha)\in E} (n_\beta n_\alpha + n_\alpha q + 2n_\beta q)}$$

bytes.

Proof. The three terms count the matrix block, the source values, and the read plus write traffic for the target accumulation. The model counts these quantities independently for every active edge. □

Definition 3.7 (Arithmetic intensity). The modeled arithmetic intensity is

$$\mathcal{I} = \frac{\text{flop}}{\text{bytes}}.$$

THEOREM 3.8 (Roofline lower bound). *Suppose a device sustains at most P modeled operations per second and at most W bytes per second. A computation that requires F operations and B bytes satisfies*

$$\boxed{T \geq \max\left\{\frac{F}{P}, \frac{B}{W}\right\}.}$$

Equivalently, its work rate is at most

$$\min\{P, W\mathcal{I}\}.$$

Proof. The operation capacity gives $T \geq F/P$. The bandwidth capacity gives $T \geq B/W$. Both restrictions hold at once. □

The roofline model is a guide rather than an exact execution law [26]. Packet structure changes both F and B. An irregular representation may remove arithmetic while lowering intensity enough to become bandwidth or launch limited.

COROLLARY 3.9 (Capacity threshold). *Let a device have available memory M bytes and scalar size s. The matrix representation alone can fit only when*

$$n \leq \sqrt{M/s}.$$

The equal packet BDLR representation alone fits when

$$n(b + 2r)s \leq M.$$

Proof. Substitute the dense and BDLR storage counts into the capacity inequality. □

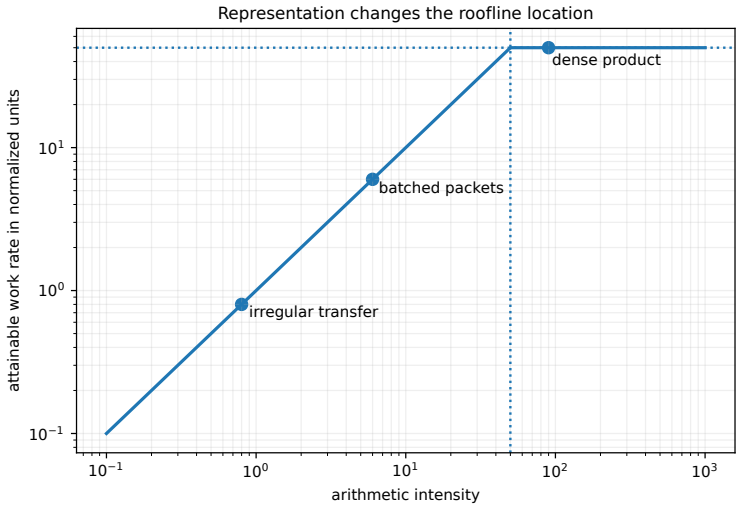


Figure 4. A representation can move the same mathematical problem between memory limited and compute limited regimes.

PROPOSITION 3.10 (Materialization cost). *Materializing $D + UV^*$ as an explicit matrix requires storage for n^2 scalars and modeled arithmetic*

$$2n^2r + O(n^2)$$

when the low rank correction is formed by dense multiplication.

Proof. The product UV^* multiplies an n by r matrix by an r by n matrix. Adding the diagonal part changes only the lower order term. □

4. **Structured packet primitives**

The packet representation can store different exact local operator types. This section records the finite formulas used by several measured suites. The results concern representation and arithmetic. They do not assert that one implementation is fastest at every size.

PROPOSITION 4.1 (Equal block banded count). *Let $p = n/b$ equal packets have width b. Suppose a block is active precisely when*

$$|i - j| \le w, \qquad 0 \le w \le p - 1.$$

The number of active blocks is

$$\boxed{N_{p,w} = p(2w + 1) - w(w + 1).}$$

For q right hand sides, the modeled application cost and scalar storage are

$$\boxed{2qb^2N_{p,w} \quad \textit{and} \quad b^2N_{p,w}.}$$

Proof. The diagonal contributes p blocks. At offset k, where $1 \le k \le w$, the upper and lower block diagonals contribute $2(p-k)$ blocks. Therefore

$$p + 2\sum_{k=1}^{w}(p-k) = p(2w+1) - w(w+1).$$

The cost and storage follow from theorem 3.1. □

PROPOSITION 4.2 (Sparse plus low rank application). *Let*

$$A = S + UV^*,$$

where S has z stored scalar nonzeros and $U, V \in \mathbb{C}^{n\times r}$. In the model that charges two operations per stored sparse entry and right hand side, applying A to q right hand sides costs

$$\boxed{2q(z+2nr)}$$

modeled operations and stores

$$\boxed{z+2nr}$$

scalar values, apart from sparse index metadata.

Proof. The sparse product costs $2zq$. The two dense thin products cost $2nrq$ each. The stored values are the nonzeros of S and the entries of the two factors. □

THEOREM 4.3 (Circulant spectral representation). *Let $c \in \mathbb{C}^n$, with indices taken modulo n, and define*

$$C(c)_{jk} = c_{j-k}.$$

Let F be the unitary discrete Fourier matrix

$$F_{jk} = \frac{1}{\sqrt{n}} \exp\left(-\frac{2\pi \mathrm{i} jk}{n}\right).$$

Then

$$\boxed{C(c) = F^* \operatorname{diag}(\sqrt{n}\, Fc) F.}$$

The operator is determined by n scalars and can be applied through two Fourier transforms and one pointwise product.

Proof. For each frequency k, let

$$v_j^{(k)} = \frac{1}{\sqrt{n}} \exp\left(\frac{2\pi \mathrm{i} jk}{n}\right).$$

Then

$$(C(c)v^{(k)})_j = \sum_{r=0}^{n-1} c_r v_{j-r}^{(k)} = \left(\sum_{r=0}^{n-1} c_r \exp\left(-\frac{2\pi \mathrm{i} rk}{n}\right)\right) v_j^{(k)}.$$

The eigenvalue is $\sqrt{n}(Fc)_k$. The vectors $v^{(k)}$ are the columns of F^*, which gives the formula. □

The fast Fourier transform evaluates the two transforms in $O(n \log n)$ arithmetic for the usual composite lengths [7]. This is an algorithmic factorization of the exact identity in theorem 4.3.

COROLLARY 4.4 (Toeplitz circulant embedding). *Let $T \in \mathbb{C}^{n\times n}$ be Toeplitz, with first column*

$$(c_0, c_1, \ldots, c_{n-1})^T$$

and first row

$$(c_0, r_1, \ldots, r_{n-1}).$$

Set

$$g = (c_0, c_1, \ldots, c_{n-1}, 0, r_{n-1}, \ldots, r_1)^T \in \mathbb{C}^{2n}$$

and append n zeros to $x \in \mathbb{C}^n$. The first n entries of the circulant product $C(g)(x, 0)^T$ equal Tx.

Proof. For $0 \le j < n$, the circular convolution sum separates into indices $k \le j$, where the coefficient is c_{j-k}, and indices $k > j$, where the wrapped coefficient is r_{k-j}. The padded coordinates do not contribute. The resulting sum is exactly the jth Toeplitz row applied to x. □

THEOREM 4.5 (Kronecker modewise application). *Let $A \in \mathbb{C}^{m\times m}$, $B \in \mathbb{C}^{n\times n}$, and $X \in \mathbb{C}^{n\times m}$. With column major vectorization,*

$$\boxed{(A \otimes B)\,\mathrm{vec}(X) = \mathrm{vec}(BXA^T).}$$

The right side has modeled cost

$$\boxed{2n^2m + 2nm^2}$$

and stores $n^2 + m^2$ factor entries instead of n^2m^2 entries.

Proof. The (i, j) entry of BXA^T is

$$\sum_{k=1}^{n}\sum_{\ell=1}^{m} B_{ik}X_{k\ell}A_{j\ell}.$$

This is the coordinate formula for $(A \otimes B)\,\mathrm{vec}(X)$. Multiplying first by B and then by A^T gives the stated count. □

COROLLARY 4.6 (Equal tensor factors). *A product of k square factors of order d, acting on a tensor with d^k entries, can be applied by k modewise contractions with modeled cost*

$$\boxed{2kd^{k+1}}$$

and factor storage kd^2.

Proof. For one mode, there are d^{k-1} fibers of length d. Multiplying each fiber by a d by d matrix costs $2d^2$, so one mode costs $2d^{k+1}$. Sum over the k modes. □

THEOREM 4.7 (Walsh Hadamard recursion). *Let $N = 2^m$, and define*

$$H_1 = [1], \qquad H_{2N} = \begin{bmatrix} H_N & H_N \\ H_N & -H_N \end{bmatrix}.$$

Then

$$H_N H_N^T = NI.$$

The product $H_N x$ can be evaluated in exactly $N \log_2 N$ additions or subtractions through the recursion.

Proof. The orthogonality identity follows by induction from block multiplication. One recursive stage forms $x_0 + x_1$ and $x_0 - x_1$, which uses N additions or subtractions, and then applies two transforms of size $N/2$. Thus

$$T(N) = 2T(N/2) + N, \qquad T(1) = 0,$$

whose solution is $T(N) = N \log_2 N$. □

5. **Exact low rank algebra and recompression**

5.1. *Products and inverses.*

THEOREM 5.1 (Exact BDLR product). *Let*

$$A = D_A + U_A V_A^*, \qquad B = D_B + U_B V_B^*$$

use the same diagonal packet layout. Then

$$AB = D_A D_B + U_C V_C^*,$$

where

$$U_C = \begin{bmatrix} D_A U_B + U_A (V_A^* U_B) & U_A \end{bmatrix}$$

and

$$V_C = \begin{bmatrix} V_B & D_B^* V_A \end{bmatrix}.$$

The unrecompressed correction rank is at most $r_A + r_B$.

Proof. Expand the product:

$$AB = D_A D_B + D_A U_B V_B^* + U_A V_A^* D_B + U_A (V_A^* U_B) V_B^*.$$

The second and fourth terms share the right factor V_B^*. The third term equals

$$U_A (D_B^* V_A)^*.$$

Concatenation gives the stated factors. □

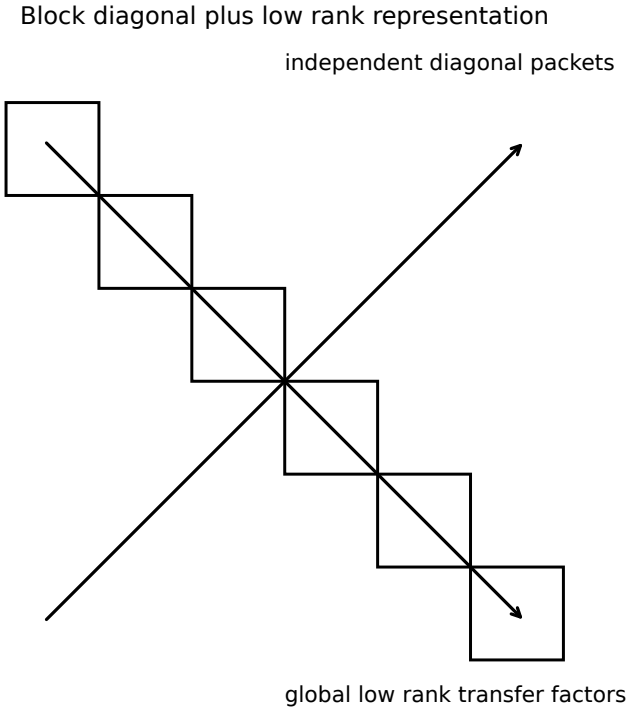


Figure 5. A regular diagonal packet structure is coupled by global low rank factors.

COROLLARY 5.2 (Rank growth in a product chain). *Let*

$$A_j = D_j + U_j V_j^*, \qquad j = 1, \dots, m,$$

share one diagonal packet layout. Before recompression, the product

$$A_m \cdots A_1$$

has a BDLR representation whose correction rank is at most

$$\boxed{\sum_{j=1}^{m} r_j.}$$

Proof. Apply theorem 5.1 inductively. Each new factor adds at most its own correction rank. □

THEOREM 5.3 (Woodbury inversion). *Assume D and*

$$S = I + V^* D^{-1} U$$

are invertible. Then

$$\boxed{(D+UV^*)^{-1} = D^{-1} - D^{-1}US^{-1}V^*D^{-1}.}$$

Proof. Multiply the proposed inverse by $D+UV^*$. The coefficient of V^*D^{-1} is

$$U - US^{-1}(I + V^*D^{-1}U) = 0.$$

The remaining term is the identity. The right inverse calculation is the same. □

The formula and its history are discussed in [15]. It reduces one large solve to packet solves with D and one dense solve with the small matrix S.

COROLLARY 5.4 (Inverse norm bound). *Under the hypotheses of theorem* 5.3,

$$\left\|(D+UV^*)^{-1}\right\| \le \left\|D^{-1}\right\| + \left\|D^{-1}U\right\| \left\|S^{-1}\right\| \left\|V^*D^{-1}\right\|.$$

Proof. Apply the triangle inequality and submultiplicativity to the Woodbury formula. □

5.2. *Optimal factor recompression.*

Standard Fact 5.5 (Eckart Young Mirsky theorem). If M has singular values

$$\sigma_1 \ge \sigma_2 \ge \cdots,$$

then its truncated singular value decomposition M_k is a best rank at most k approximation. It satisfies

$$\|M - M_k\|_2 = \sigma_{k+1}$$

and

$$\|M - M_k\|_{\mathrm{F}} = \left(\sum_{j>k} \sigma_j^2\right)^{1/2}.$$

This is classical [11] and [21]. The next theorem shows that an n dimensional singular value decomposition is unnecessary when a correction is already factored.

THEOREM 5.6 (Small core recompression). *Let*

$$L \in \mathbb{C}^{m\times r}, \qquad R \in \mathbb{C}^{n\times r}.$$

Take compact QR factorizations

$$L = Q_L R_L, \qquad R = Q_R R_R,$$

where the columns of Q_L and Q_R are orthonormal. Form the small core

$$G = R_L R_R^*$$

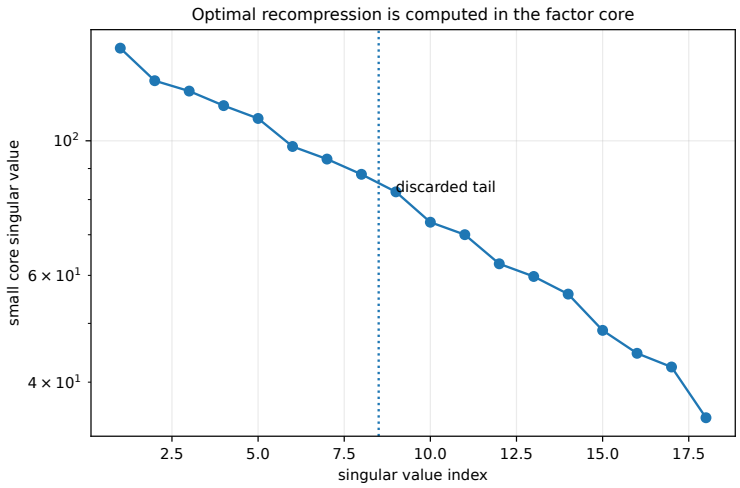


Figure 6. The approximation decision is made from the singular values of a small core whose dimension is the unrecompressed rank.

and let

$$G = W\Sigma Z^*$$

be a singular value decomposition. Define

$$L_k = Q_L W_k \Sigma_k^{1/2}, \qquad R_k = Q_R Z_k \Sigma_k^{1/2}.$$

Then

$$L_k R_k^*$$

is a best rank at most k approximation to LR^*. Its spectral and Frobenius errors are the singular value tails in standard fact 5.5*. The core dimensions equal the column ranks of the two input factors. A noncompact reduced QR may retain extra zero singular directions without changing the conclusion.*

Proof. The exact factor product is

$$LR^* = Q_L G Q_R^*.$$

The two outer maps are isometries on their column spaces. They preserve the nonzero singular values and every unitarily invariant norm of the core. Truncating the singular value decomposition of G and applying standard fact 5.5 gives the result. The compact QR dimensions equal the ranks of the factor column spaces. □

Remark 5.7. The large matrices enter through QR factorizations of the factors. The singular value problem has dimension at most the unrecompressed

rank. Updating and recompressing low rank factors is standard numerical linear algebra [13] and [4].

THEOREM 5.8 (Product chain perturbation). *Let $A_1, \dots, A_m$ be exact factors and let*

$$\widehat{A}_j = A_j + E_j.$$

Then

$$\boxed{\left\| \widehat{A}_m \cdots \widehat{A}_1 - A_m \cdots A_1 \right\| \le \sum_{j=1}^{m} \left(\prod_{i=j+1}^{m} \left\| \widehat{A}_i \right\| \right) \|E_j\| \left(\prod_{i=1}^{j-1} \|A_i\| \right).}$$

Empty products are one.

Proof. Use the telescoping identity

$$\widehat{A}_m \cdots \widehat{A}_1 - A_m \cdots A_1 = \sum_{j=1}^{m} \widehat{A}_m \cdots \widehat{A}_{j+1} E_j A_{j-1} \cdots A_1.$$

Take norms term by term. □

THEOREM 5.9 (Certified repeated recompression). *Let*

$$P_0 = I, \qquad P_j = A_j P_{j-1},$$

be exact partial products. Suppose an implemented chain satisfies

$$\widehat{P}_0 = I, \qquad \widehat{P}_j = A_j \widehat{P}_{j-1} + E_j,$$

where E_j is the additive error introduced after the jth multiplication, for example by factor recompression. Then

$$\boxed{\left\| \widehat{P}_m - P_m \right\| \le \sum_{j=1}^{m} \left(\prod_{i=j+1}^{m} \|A_i\| \right) \|E_j\|.}$$

If $\|E_j\| \le \delta_j$, the same formula with δ_j gives a certified final error.

Proof. Set $D_j = \widehat{P}_j - P_j$. Then

$$D_j = A_j D_{j-1} + E_j, \qquad D_0 = 0.$$

Repeated substitution gives

$$D_m = \sum_{j=1}^{m} A_m A_{m-1} \cdots A_{j+1} E_j.$$

Taking norms yields the result. □

6. **Sparse transfer graphs and schedules**

6.1. *Support under algebraic operations.*

THEOREM 6.1 (Path support of powers). *For every positive integer m,*

$$P_\gamma A^m P_\alpha$$

is the sum over directed paths

$$\alpha = \alpha_0 \longrightarrow \alpha_1 \longrightarrow \cdots \longrightarrow \alpha_m = \gamma$$

in G_A, with contribution

$$A_{\alpha_m \alpha_{m-1}} \cdots A_{\alpha_1 \alpha_0}.$$

If no such path exists, the block vanishes.

Proof. Insert the packet identity between every two consecutive factors of A. Every nonzero term determines one path of length m. □

COROLLARY 6.2 (Polynomial support). *If*

$$p(z) = \sum_{j=0}^{d} c_j z^j,$$

then

$$\operatorname{supp}(p(A))$$

is contained in the union of the support relations of A^j for $0 \le j \le d$.

Proof. Apply theorem 6.1 to every power in the polynomial. □

COROLLARY 6.3 (Acyclic transfer nilpotence). *Let K contain only strict transfer blocks. If its support graph is acyclic and its longest directed path has h edges, then*

$$K^{h+1} = 0.$$

Proof. There is no directed path of length $h + 1$. Every block of K^{h+1} therefore vanishes by theorem 6.1. □

6.2. *Block triangular solves.*

Definition 6.4 (Topological level). Let G be a directed acyclic graph. The first level contains its source vertices. Remove them. The next level contains the new sources. Continue until every vertex has been assigned.

THEOREM 6.5 (Level schedule for a triangular solve). *Let $A = D + K$ be block lower triangular after a topological ordering. Assume every diagonal block D_α is invertible. The solve*

$$Ax = b$$

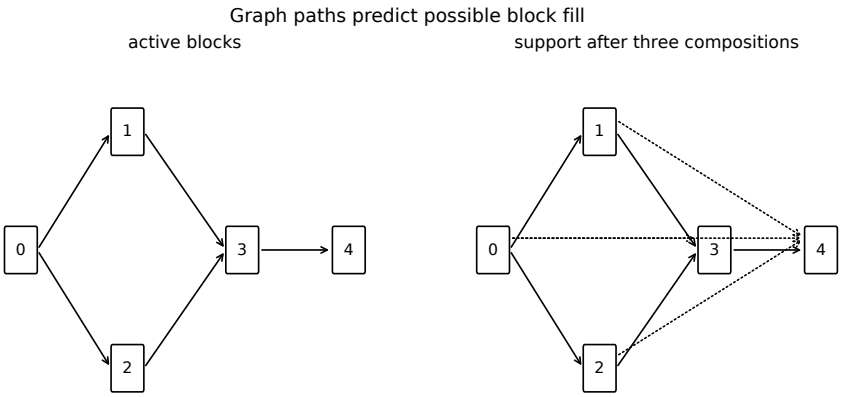


Figure 7. Composition creates possible support along directed paths. Cancellation can remove an allowed block, so the graph gives an inclusion rather than an equality.

can be completed in one sequential stage per topological level. Blocks in the same level may be solved independently after all earlier levels are known.

Proof. The equation in packet β is

$$D_\beta x_\beta = b_\beta - \sum_{\alpha \to \beta} K_{\beta\alpha} x_\alpha.$$

Every predecessor of β lies in an earlier level. Once those values are available, x_β is obtained from the local solve with D_β. Vertices in the same level have no directed edge between them, so their solves are independent. □

COROLLARY 6.6 (Sequential depth). *The number of sequential stages in the level schedule equals one plus the length of the longest directed path.*

Proof. The recursive level of a vertex is one plus the largest level of its predecessors. The largest level therefore counts the vertices on a longest path. □

6.3. *Fill under elimination.*

THEOREM 6.7 (Inverse support lies in the transitive closure). *Let A be an invertible packet operator. If there is no directed path from packet α to packet β in G_A, then*

$$\boxed{P_\beta A^{-1} P_\alpha = 0.}$$

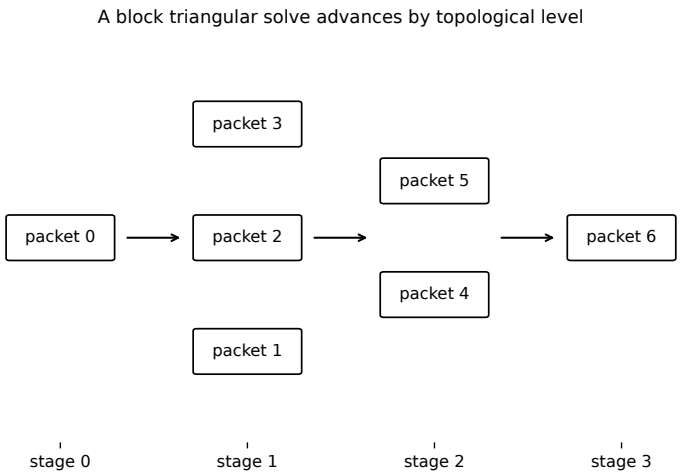


Figure 8. Packets in the same topological level can be processed concurrently in a triangular solve.

Proof. Let R be the set of packets reachable from α, including α, and set

$$X_R = \bigoplus_{\gamma \in R} X_\gamma.$$

No active edge leaves R, so $AX_R \subseteq X_R$. The restriction of A to the finite dimensional space X_R is injective. It is therefore onto X_R. Hence $A^{-1}X_R \subseteq X_R$. Since $\beta \notin R$, the stated block of A^{-1} vanishes. □

Proposition 6.8 (Schur support inclusion). *Split the packets into retained indices R and hidden indices H. Suppose the hidden block A_{HH} is invertible. The retained Schur complement is*

$$S = A_{RR} - A_{RH}A_{HH}^{-1}A_{HR}.$$

A retained block (β, α) can be active only if it was already active in A_{RR}, or if there is a directed walk from α through hidden packets to β.

Proof. The correction block is

$$P_\beta A_{RH}A_{HH}^{-1}A_{HR}P_\alpha.$$

It can be nonzero only when $A_{HR}P_\alpha$ enters a hidden packet, the inverse hidden operator connects hidden packets, and $P_\beta A_{RH}$ exits to β. By theorem 6.7, the hidden inverse can connect only packets joined by a hidden directed path. Cancellation may still remove an allowed block. □

Sparse elimination and fill are treated systematically in [9]. The packet graph records the same structural issue at block scale.

7. **Krylov methods and packet preconditioning**

Definition 7.1 (Krylov space). For a linear operator A, a vector v, and a positive integer m, define

$$\mathcal{K}_m(A, v) = \operatorname{span}\{v, Av, \dots, A^{m-1}v\}.$$

Proposition 7.2 (Invariant packet Krylov decomposition). *Assume*

$$[A, P_\alpha] = 0$$

for every packet. If

$$v = \sum_\alpha v_\alpha, \qquad v_\alpha = P_\alpha v,$$

then

$$A^j v = \sum_\alpha A_\alpha^j v_\alpha$$

for every $j \geq 0$, where

$$A_\alpha = P_\alpha A P_\alpha.$$

Hence

$$\mathcal{K}_m(A, v) \subseteq \bigoplus_\alpha \mathcal{K}_m(A_\alpha, v_\alpha).$$

Proof. Commutation gives

$$A^j P_\alpha = P_\alpha A^j.$$

Apply this identity to each packet component of v. □

Theorem 7.3 (Approximate packet Krylov bound). *Write*

$$A = D + K,$$

where D is packet diagonal. For every positive integer j,

$$\boxed{\left\| A^j - D^j \right\| \leq j M^{j-1} \|K\|,}$$

where

$$M = \max\{\|A\|, \|D\|\}.$$

Consequently,

$$\operatorname{dist}(A^j v, \bigoplus_\alpha \mathcal{K}_{j+1}(D_\alpha, v_\alpha)) \leq j M^{j-1} \|K\| \|v\|.$$

Proof. Use the telescoping identity

$$A^j - D^j = \sum_{k=0}^{j-1} A^{j-1-k} K D^k.$$

Each term has norm at most $M^{j-1}\|K\|$. The vector $D^j v$ lies in the direct sum of local Krylov spaces. □

THEOREM 7.4 (Block Jacobi condition bound). *Let*

$$A = D + K$$

be Hermitian positive definite with positive block diagonal part D. Suppose

$$E = D^{-1/2} K D^{-1/2}$$

is Hermitian and

$$\|E\|_2 \leq \eta < 1.$$

Then

$$\boxed{\operatorname{cond}_2(D^{-1/2} A D^{-1/2}) \leq \frac{1+\eta}{1-\eta}.}$$

Proof. For every unit vector x,

$$|x^* E x| \leq \eta.$$

The Rayleigh quotient of $I + E$ lies in $[1-\eta, 1+\eta]$. The quotient of the extreme eigenvalue bounds gives the result. □

Standard Fact 7.5 (Preconditioned conjugate gradient convergence). Let A and M be Hermitian positive definite, and set

$$\kappa = \operatorname{cond}_2(M^{-1/2} A M^{-1/2}).$$

The energy norm error of preconditioned conjugate gradient satisfies

$$\|x_m - x_*\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^m \|x_0 - x_*\|_A .$$

This standard estimate is discussed in [24]. Combining it with theorem 7.4 yields an explicit iteration certificate when the normalized cross packet coupling is small.

COROLLARY 7.6 (Certified packet preconditioning). *Under the hypotheses of theorem* 7.4, *the convergence factor in standard fact* 7.5 *is bounded by the value obtained from*

$$\kappa = \frac{1+\eta}{1-\eta}.$$

Proof. Insert the condition bound from theorem 7.4 into the monotone convergence factor in standard fact 7.5. □

8. **Packetwise mixed precision**

8.1. *One operator.*

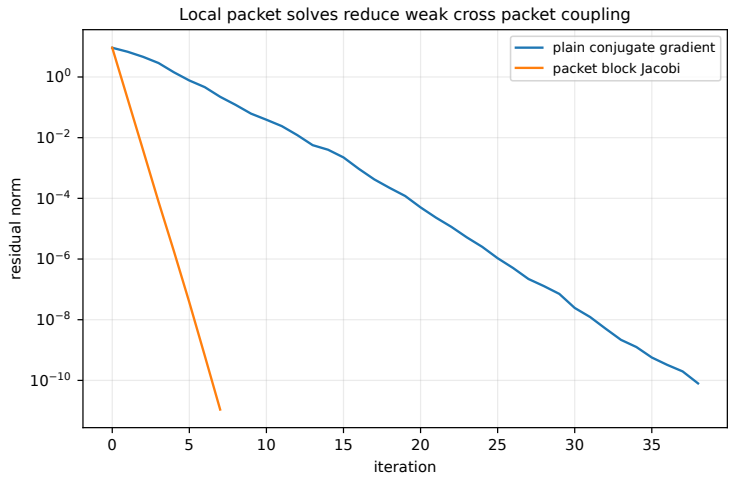


Figure 9. The block diagonal packet operator serves as a reusable local preconditioner in a weakly coupled positive system.

THEOREM 8.1 (Packet perturbation certificate). *Let*

$$A = \sum_{e \in E} A_e, \qquad \widehat{A} = \sum_{e \in E} (A_e + E_e),$$

where edge e starts at packet $\alpha(e)$, and assume

$$A_e = A_e P_{\alpha(e)}, \qquad E_e = E_e P_{\alpha(e)}.$$

Then

$$\boxed{\left\| (\widehat{A} - A)x \right\| \leq \sum_{e \in E} \|E_e\| \left\| P_{\alpha(e)} x \right\|.}$$

If

$$\|E_e\| \leq \varepsilon_e \|A_e\|,$$

then

$$\boxed{\left\| (\widehat{A} - A)x \right\| \leq \sum_{e \in E} \varepsilon_e \|A_e\| \left\| P_{\alpha(e)} x \right\|.}$$

Proof. Exact packet evaluation gives

$$(\widehat{A} - A)x = \sum_{e} E_e P_{\alpha(e)} x.$$

Apply the triangle inequality and the operator norm bound to each term. □

Standard Fact 8.2 (Floating point matrix product model). Under the standard floating point model, a computed matrix product may be written as an exact product with a dimension dependent backward perturbation. The

perturbation constant is controlled by the unit roundoff and the inner dimension when overflow and underflow are excluded.

Detailed forms and their hypotheses are given in [17]. Packetwise use of this fact produces edge bounds ε_e for theorem 8.1.

8.2. *Several operators.*

Corollary 8.3 (Mixed precision product chain). *If each factor A_j is replaced by a packetwise approximation $\widehat{A}_j$, then the final operator error is bounded by theorem* 5.8. *Each $\|E_j\|$ may in turn be bounded by the packet certificate in theorem* 8.1.

Proof. Apply theorem 8.1 at every stage and substitute the resulting operator error bounds into theorem 5.8. □

8.3. *Continuous allocation.*

Theorem 8.4 (Continuous precision allocation). *Let $s_e > 0$ be sensitivities and $c_e > 0$ be costs. Consider*

$$\min_{\varepsilon_e>0} \sum_e c_e \log \frac{1}{\varepsilon_e}$$

subject to

$$\sum_e s_e \varepsilon_e = \tau.$$

The unique minimizer is

$$\boxed{\varepsilon_e = \frac{\tau c_e}{s_e \sum_f c_f}.}$$

Proof. The Lagrangian derivative gives

$$-\frac{c_e}{\varepsilon_e} + \lambda s_e = 0.$$

Thus

$$\varepsilon_e = \frac{c_e}{\lambda s_e}.$$

The constraint gives

$$\lambda = \frac{\sum_f c_f}{\tau}.$$

The objective is strictly convex on the positive orthant, so the stationary point is the unique minimizer. □

Theorem 8.5 (Clipped precision allocation). *Add upper bounds*

$$0 < \varepsilon_e \le m_e.$$

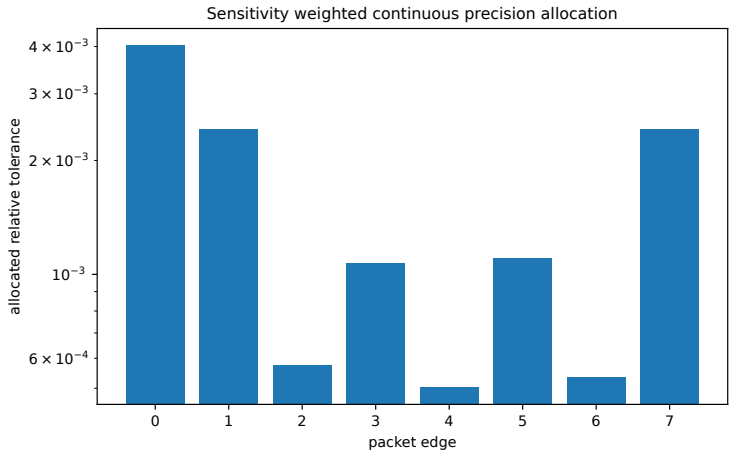


Figure 10. More sensitive packet edges receive tighter continuous tolerances.

Assume

$$0 < \tau \leq \sum_e s_e m_e.$$

There is a unique $\lambda > 0$*, except at the maximal tolerance endpoint, such that*

$$\boxed{\varepsilon_e = \min\left\{m_e, \frac{c_e}{\lambda s_e}\right\}}$$

and

$$\sum_e s_e \varepsilon_e = \tau.$$

This vector minimizes the same objective over the clipped feasible set.

Proof. The Karush Kuhn Tucker conditions give the unconstrained form on every nonactive coordinate and equality with m_e on every active upper bound. The weighted sum

$$\phi(\lambda) = \sum_e s_e \min\left\{m_e, \frac{c_e}{\lambda s_e}\right\}$$

is continuous and nonincreasing. It decreases from the maximal feasible value to zero. The constraint determines λ. Strict convexity gives uniqueness of the minimizer. □

8.4. *Discrete data types.*

Definition 8.6 (Discrete precision option). For edge e, a discrete precision option k has a measured or modeled cost $c_{e,k}$ and a nonnegative integer error weight $w_{e,k}$. Exactly one option must be selected on every edge.

THEOREM 8.7 (Exact discrete precision program). *Let the total integer error budget be B. Define*

$$F_j(b)$$

as the minimum cost for assigning precisions to the first j edges with total weight exactly b. Then

$$F_0(0) = 0, \qquad F_0(b) = +\infty \quad \text{for } b \neq 0.$$

Set $F_{j-1}(u) = +\infty$ whenever $u < 0$ or the state is unreachable. Then

$$\boxed{F_j(b) = \min_k \{F_{j-1}(b - w_{j,k}) + c_{j,k}\}.}$$

The minimum of $F_m(b)$ over $0 \leq b \leq B$ is the exact minimum cost for the quantized model.

Proof. Every feasible assignment for the first j edges ends with one option k on edge j. Removing it leaves an optimal or suboptimal assignment for the first $j-1$ edges with residual weight $b - w_{j,k}$. Replacing a suboptimal prefix by an optimal one can only lower cost. The recurrence therefore has optimal substructure. Induction on j proves the result. □

Remark 8.8. The discrete program is exact only for the supplied options and quantized error weights. Real kernels also depend on accumulation type, scaling, overflow, underflow, tensor core availability, and shape restrictions. Mixed precision iterative refinement has its own conditioning requirements [5].

9. **Discovery and exact packet compilation**

9.1. *Exact spectral discovery.*

Standard Fact 9.1 (Simultaneous spectral decomposition). A finite family of pairwise commuting normal matrices admits a common orthonormal basis of joint eigenvectors. The joint eigenspaces associated with distinct eigenvalue tuples give the coarsest orthogonal packet system on which every matrix in the family acts as a scalar.

PROPOSITION 9.2 (Exact block discovery certificate). *For an orthogonal packet system (P_α), define the block diagonal compression*

$$\Pi_{\mathcal{P}}(A) = \sum_\alpha P_\alpha A P_\alpha.$$

Then

$$\boxed{\|A - \Pi_{\mathcal{P}}(A)\|_{\mathrm{F}}^2 = \sum_{\alpha \neq \beta} \|P_\beta A P_\alpha\|_{\mathrm{F}}^2.}$$

In operator norm,

$$\boxed{\|A - \Pi_{\mathcal{P}}(A)\|_2 \leq \sum_{\alpha \neq \beta} \|P_\beta A P_\alpha\|_2 .}$$

Thus any proposed packet discovery has an exact finite residual certificate after its projectors are known.

Proof. Distinct matrix blocks are orthogonal in the Hilbert Schmidt inner product. Their squared Frobenius norms therefore add. The operator norm estimate follows from the triangle inequality. □

This supplies one exact discovery regime and one exact a posteriori certificate. Perturbation of spectral packets requires a gap. The Davis Kahan theorem gives quantitative control for Hermitian spectral subspaces under such a gap [8]. Randomized low rank discovery supplies another controlled regime [16] and [20]. Neither result gives globally optimal packet discovery for arbitrary operators.

9.2. *Candidate packet trees.*

Definition 9.3 (Candidate representation tree). A candidate representation tree is a finite rooted tree. Every node v has

$$\kappa_v \geq 0, \qquad \epsilon_v \geq 0,$$

representing the cost and error of keeping v as one leaf. Every internal node also has a split overhead $\sigma_v \geq 0$. An admissible pruning contains exactly one selected node on every root to terminal path.

The cost and error of a pruning $\mathcal{L}$ are

$$\operatorname{cost}(\mathcal{L}) = \sum_{v \in \mathcal{L}} \kappa_v + \sum_{u \text{ split}} \sigma_u$$

and

$$\operatorname{err}(\mathcal{L}) = \sum_{v \in \mathcal{L}} \epsilon_v.$$

The model can include arithmetic, storage, launch, transfer, and precision costs inside κ_v. It remains exact only when the declared totals are additive.

THEOREM 9.4 (Exact Lagrangian packet compiler). *Fix* $\lambda \geq 0$ *and define*

$$J_\lambda(\mathcal{L}) = \operatorname{cost}(\mathcal{L}) + \lambda \operatorname{err}(\mathcal{L}).$$

For every node v, *set*

$$K_\lambda(v) = \kappa_v + \lambda \epsilon_v$$

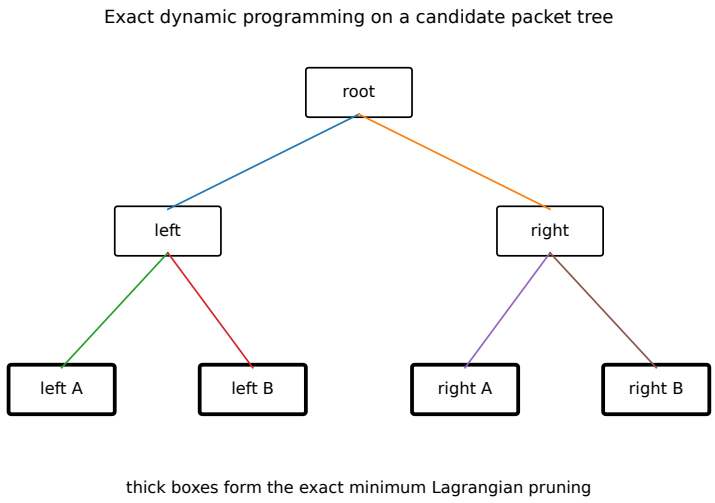


Figure 11. The thick leaves are selected by the exact Lagrangian recurrence.

and

$$S_\lambda(v) = \sigma_v + \sum_{w \in \mathrm{child}(v)} F_\lambda(w).$$

For a terminal node, set

$$F_\lambda(v) = K_\lambda(v).$$

For an internal node, the recurrence

$$\boxed{F_\lambda(v) = \min\{K_\lambda(v), S_\lambda(v)\}}$$

computes the exact minimum Lagrangian objective over every admissible pruning of the subtree rooted at v.

Proof. A pruning of the subtree rooted at v has two possible forms. It keeps v, in which case its objective is $K_\lambda(v)$. Otherwise it splits v and independently chooses an admissible pruning in every child subtree. By induction, the optimal child values are $F_\lambda(w)$. The split objective is therefore $S_\lambda(v)$. Taking the smaller value is exact. □

This recurrence is a computational version of best basis selection and Bellman optimality [3] and [6]. The node costs here may include operator application, transfer, launch, precision, and output conversion rather than only a signal entropy.

Corollary 9.5 (Shared compilation for an operator family). *Let operators $A^{(1)}, \ldots, A^{(m)}$ use the same candidate tree. Give operator j a nonnegative reuse*

weight r_j. Replace the local keep cost and error by

$$\kappa_v = \kappa_v^{\mathrm{setup}} + \sum_{j=1}^{m} r_j \kappa_v^{(j)}$$

and

$$\epsilon_v = \sum_{j=1}^{m} r_j \epsilon_v^{(j)}.$$

Then theorem 9.4 gives the exact optimal shared pruning for the additive weighted objective.

Proof. The weighted family objective is still the sum of local leaf and split terms. Apply theorem 9.4 to the aggregated quantities. □

PROPOSITION 9.6 (Monotone Lagrangian frontier). *Let $0 \le \lambda_1 < \lambda_2$. Choose optimal prunings $\mathcal{L}_1$ and $\mathcal{L}_2$ for the two Lagrangian parameters. Then*

$$\boxed{\operatorname{err}(\mathcal{L}_2) \le \operatorname{err}(\mathcal{L}_1)}$$

and

$$\boxed{\operatorname{cost}(\mathcal{L}_2) \ge \operatorname{cost}(\mathcal{L}_1).}$$

Proof. Optimality gives

$$\operatorname{cost}(\mathcal{L}_1) + \lambda_1 \operatorname{err}(\mathcal{L}_1) \le \operatorname{cost}(\mathcal{L}_2) + \lambda_1 \operatorname{err}(\mathcal{L}_2)$$

and

$$\operatorname{cost}(\mathcal{L}_2) + \lambda_2 \operatorname{err}(\mathcal{L}_2) \le \operatorname{cost}(\mathcal{L}_1) + \lambda_2 \operatorname{err}(\mathcal{L}_1).$$

Adding the inequalities yields

$$(\lambda_2 - \lambda_1)\big(\operatorname{err}(\mathcal{L}_2) - \operatorname{err}(\mathcal{L}_1)\big) \le 0.$$

The error ordering follows. Substitution into the first inequality gives the cost ordering. □

COROLLARY 9.7 (Supported Pareto points). *For $\lambda > 0$, every Lagrangian optimum is Pareto optimal for the pair*

$$(\mathrm{cost}, \mathrm{err}).$$

The quantized budget compiler may recover Pareto points that are not optimal for any Lagrangian parameter.

Proof. If another pruning had no larger cost and no larger error, with one strict inequality, its Lagrangian objective would be smaller. This contradicts optimality. A finite nonconvex frontier can contain points that are not supported by a line of slope $-\lambda$, so the budget formulation is genuinely additional. □

9.3. *An exact error budget compiler.*

THEOREM 9.8 (Quantized budget dynamic program). *Assume every keep error is a nonnegative integer w_v. Let $B \in \mathbb{N}$. For each node v, define $F_v(b)$ as the minimum cost of a pruning of the subtree at v with total error exactly b. The keep option contributes*

$$F_v(w_v) \leq \kappa_v.$$

The split option is obtained by min plus convolution of the child tables, followed by addition of σ_v. The smallest value among

$$F_{\mathrm{root}}(b), \qquad 0 \leq b \leq B,$$

is the exact minimum cost among prunings whose quantized error is at most B.

Proof. The keep case is immediate. In the split case, an admissible pruning is the union of one admissible pruning from each child. Its error is the sum of the child errors and its cost is the sum of their costs plus σ_v. Repeated min plus convolution enumerates every possible distribution of the integer budget across the child subtrees. Induction on the tree proves exactness. □

COROLLARY 9.9 (Pseudo polynomial complexity). *If the maximum number of children is bounded and the budget is B, the quantized compiler has pseudo polynomial complexity in the number of tree nodes and B. A direct pairwise convolution implementation has quadratic dependence on B at each merge.*

Proof. Every table has at most $B+1$ entries. Merging two tables by direct convolution examines at most $(B+1)^2$ pairs. A bounded number of merges occurs at every node. □

9.4. *Decision stability and local updates.*

Definition 9.10 (Decision gap). At an internal node v, let

$$g_v = |K_\lambda(v) - S_\lambda(v)|.$$

This is the gap between the keep and split scores after the child subproblems have been solved.

THEOREM 9.11 (Compiler stability). *Suppose the keep score and split score at every internal node change by less than $g_v/2$. Then every strict keep or split decision in the original optimal pruning remains unchanged.*

Proof. If the original keep score is lower by g_v, then after perturbation it can increase by less than $g_v/2$, while the split score can decrease by less than $g_v/2$. The order remains strict. The split case is the same. Apply the argument from the leaves upward. □

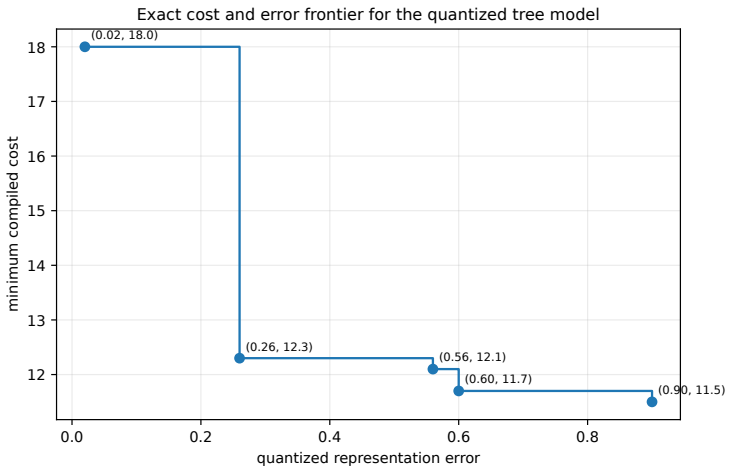


Figure 12. The quantized dynamic program produces the exact cost and error frontier for its finite model.

THEOREM 9.12 (Local recompilation). *Suppose local costs change only inside the subtree rooted at u. Then every dynamic programming value outside that subtree and outside the ancestor chain from u to the root remains unchanged. Recomputing the subtree at u and that ancestor chain gives the exact new global optimum.*

Proof. A node value depends only on its own local data and the values of its children. Nodes outside the changed subtree and outside the ancestor chain have the same local data and the same child values. Their values remain unchanged. Every affected dependency lies inside the changed subtree or on the unique path from u to the root. □

Remark 9.13 (Boundary of the compiler theorem). The exact compiler requires a finite candidate tree and additive local models. Arbitrary overlapping decompositions, nonlocal cache interactions, global rank coupling, and unrestricted structure discovery can destroy optimal substructure. No universal approximation factor is proved for those settings.

10. **Device scheduling and amortization**

10.1. *Shape classes.*

Definition 10.1 (Shape class). Two active packet blocks lie in the same shape class when their input dimension, output dimension, right hand side count, scalar type, accumulation type, and kernel contract agree.

PROPOSITION 10.2 (Launch reduction by shape). *Suppose B independent active blocks fall into g shape classes, and suppose the backend accepts one batched array or pointer list for each class. Grouped execution then requires at most g launches instead of B, without changing the arithmetic or the output.*

Proof. Place all blocks in one class along a batch dimension and invoke one batched kernel for that class. Every block receives the same local computation. One launch per class is sufficient. □

CUDA provides graph capture and replay for repeated dependency graphs [22]. These mechanisms reduce repeated launch work, but they do not change the operator or its arithmetic count.

10.2. *Critical path.*

THEOREM 10.3 (Acyclic schedule depth). *Let a packet task graph be acyclic. Assume every task takes one normalized stage, every edge is a dependency, and an unlimited number of independent tasks may run concurrently. The minimum possible number of sequential stages equals the number of vertices on a longest directed path.*

Proof. Every path gives a lower bound because its tasks must execute in order. The topological level schedule in theorem 6.5 achieves exactly this depth. □

10.3. *One time costs.*

THEOREM 10.4 (Amortization threshold). *Let $s \geq 0$ be a one time setup cost. Let t_b be the baseline time per use and t_c the candidate time per use. If $t_b > t_c$, the candidate is faster after R uses exactly when*

$$R > \frac{s}{t_b - t_c}.$$

The smallest integer reuse count is

$$\boxed{R_{\min} = \left\lfloor \frac{s}{t_b - t_c} \right\rfloor + 1.}$$

If $t_b \leq t_c$, no positive reuse count amortizes a positive setup cost.

Proof. Compare

$$s + Rt_c$$

with

$$Rt_b.$$

Rearranging gives the stated inequality. □

COROLLARY 10.5 (Graph replay threshold). *If graph capture costs c, ordinary execution costs t_o, and replay costs $t_r < t_o$, then capture is profitable when*

$$R > \frac{c}{t_o - t_r}.$$

Proof. Apply theorem 10.4 with setup cost c. □

COROLLARY 10.6 (Materialization threshold). *Suppose a compressed representation costs t_c per downstream use, an explicit representation costs t_e per use, and one time materialization costs m. Materialization is profitable only when*

$$R > \frac{m}{t_c - t_e}$$

and $t_c > t_e$.

Proof. Compare repeated compressed use with materialization followed by repeated explicit use. □

11. **Benchmark contracts and measurement protocol**

Performance data are meaningful only after the mathematical and timing contracts are fixed. The protocol follows standard concerns from numerical linear algebra, floating point analysis, and machine performance modeling [17], [14], and [26].

Definition 11.1 (Case signature). A case signature records the dimensions, packet layout, rank, right hand side count, random seed, source values, source type, reference computation, and output contract.

Definition 11.2 (Fair match). Two measured rows are a fair match when they share one case signature, return the same declared output, use the same correctness reference, and have identical timing scopes. If one row includes an extra conversion or setup cost, it is a different scope and must be reported separately or added to both rows before comparison.

PROPOSITION 11.3 (Fair matching is an equivalence relation). *On rows with complete metadata, equality of the case signature, output contract, correctness reference, and timing scope is an equivalence relation.*

Proof. Equality of each metadata field is reflexive, symmetric, and transitive. Their conjunction has the same properties. □

Definition 11.4 (Representation gain). Representation gain compares dense and structured work or storage for the same operator, independently of the processor.

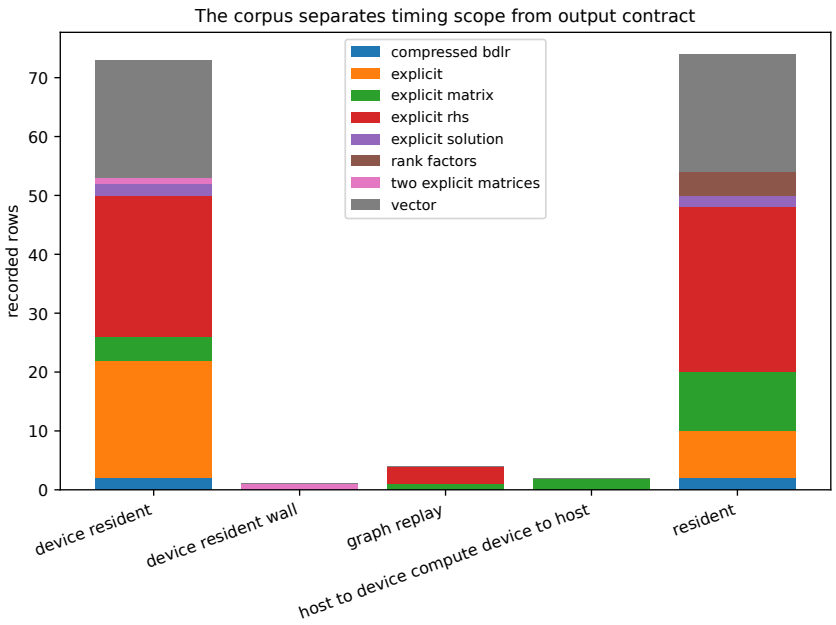


Figure 13. The corpus records timing scope and output contract as separate fields.

Definition 11.5 (Device gain). Device gain compares processors for the same representation, output, and timing scope.

Definition 11.6 (Kernel gain). Kernel gain compares two implementations on the same device for the same representation, output, and timing scope.

Definition 11.7 (Amortization gain). Amortization gain records the effect of one time discovery, compilation, transfer, capture, or materialization at a declared reuse count.

A measured total speedup can mix these four effects. A multiplicative decomposition is only exact when matched rows isolate the factors. Otherwise the decomposition is interpretive.

Remark 11.8 (Logical work rate). A structured method may report a dense equivalent logical work rate greater than the physical dense throughput because it does not execute the dense operations. This statistic describes avoided work. It is not a claim that the device performed those operations.

12. **Recorded CPU and GPU corpus**

The empirical corpus contains 154 measured rows from sixteen suites. It has 74 CPU rows and 80 GPU rows. The GPU was an NVIDIA GeForce RTX

Suite	Rows	Suite	Rows
Dense multiplication	28	BDLR application	15
Block diagonal	12	BDLR product	12
Circulant	12	Toeplitz	12
Kronecker	12	Block banded	8
Sparse plus low rank	8	HODLR	8
Strassen	6	Transfer pipeline	5
Walsh Hadamard	4	Hidden blocks	4
Randomized low rank	4	Iterative refinement	4

Table 1. Coverage of the sanitized measured corpus.

Method	Median time in ms	Logical GFLOP per second	Relative error
FP64	541.2803	253.91	2.18×10^{-15}
FP32	18.1309	7580.35	5.76×10^{-7}
TF32	8.6999	15797.75	2.94×10^{-4}
FP16	4.0264	34134.72	3.60×10^{-4}
BF16	4.1482	33132.00	2.88×10^{-3}

Table 2. Recorded dense accuracy and throughput at one dimension.

3070 Ti Laptop GPU with compute capability 8.6 and 8 GB nominal memory. The recorded software used PyTorch 2.11.0 with a CUDA 12.8 build. These facts identify the environment. They do not predict another machine.

12.1. *Dense precision frontier.* At dimension 4096, the device resident dense rows gave the values in table 2.

The frontier is not a universal ordering of scalar types. It records one generated family. Packet precision assignment should use local sensitivities and supported kernels rather than one global type.

12.2. *Regular packet operators.* For an 8192 by 8192 equal block operator with block width 64 and 32 right hand sides, the batched packet calculation took

$$0.046032 \text{ ms.}$$

It was

$$14.8919$$

times faster than the matched device resident dense GPU row and

$$235.7404$$

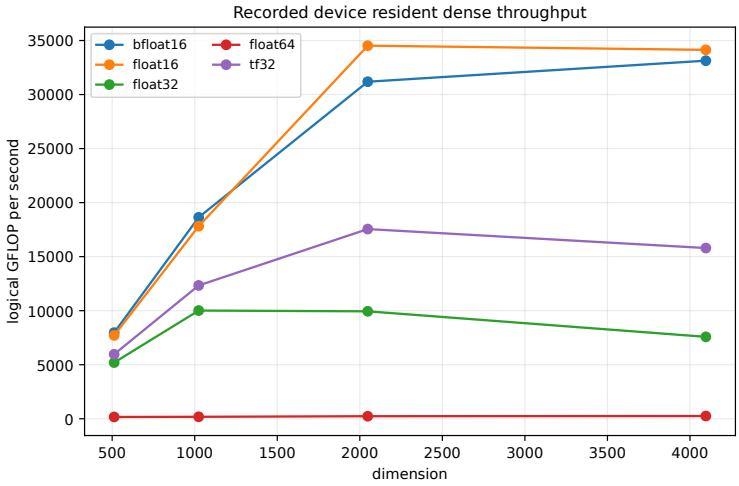


Figure 14. Recorded device resident dense throughput across dimensions and scalar types.

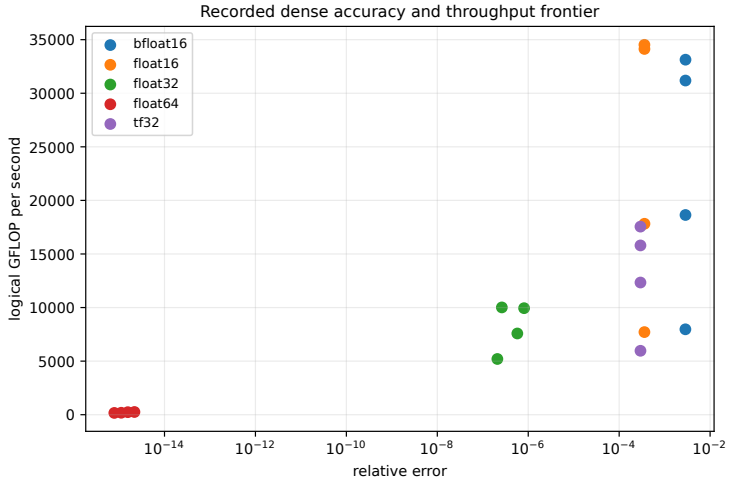


Figure 15. The speed and error tradeoff depends on both precision and problem size.

times faster than the matched CPU dense row. The relative error was

$$8.17 \times 10^{-8}.$$

For BDLR application with the same dimension and right hand side count, block width 64, and rank 8, CUDA Graph replay took

$$0.079872 \text{ ms}.$$

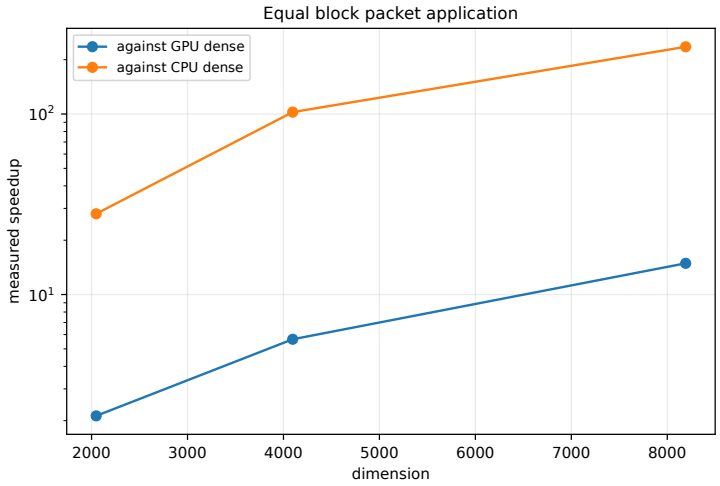


Figure 16. Regular equal shapes expose both representation reduction and batched execution.

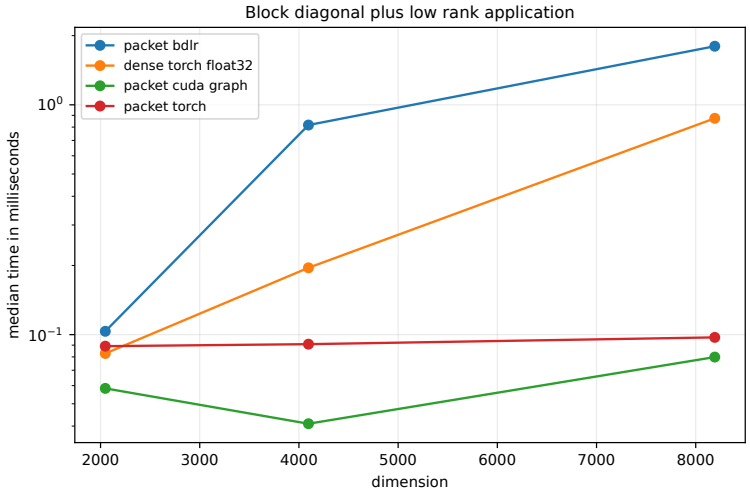


Figure 17. Representation and launch strategy both affect BDLR application time.

It was

$$10.9355$$

times faster than the matched dense GPU row and

$$153.6527$$

times faster than the matched CPU dense row. Capture time was excluded from the replay scope and must be amortized separately.

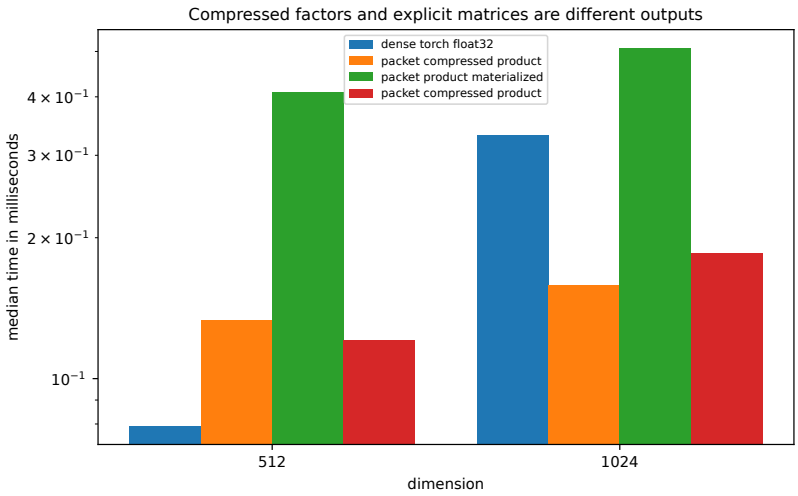


Figure 18. Compressed and explicit products answer different computational questions.

12.3. *Compressed products.* A compressed BDLR product returns block and low rank factors. Materialization returns all matrix entries. The measured product rows show the cost of changing the output contract.

The algebra in theorem 5.1 explains why the compressed product can remain small. The materialization cost in proposition 3.10 explains why explicit output can erase the advantage.

12.4. *Discovery and transfer.* For hidden permuted blocks at dimension 512, structure discovery took 2.1102 ms, dense application took 0.0969 ms, and packet application took 0.0151 ms. The exact threshold in theorem 10.4 is 26 uses. At dimension 1024, the corresponding values were 5.2726, 0.1896, and 0.0241 ms, giving 32 uses.

For one 2048 dense FP32 product, device resident graph replay took 1.9993 ms. The end to end pageable path took 7.3036 ms. The pinned path took 6.3084 ms. Two serial device resident products took 3.6557 ms, while the tested two stream path took 4.2368 ms.

12.5. *Negative controls.* The corpus retains methods that lost to their matched baselines.

- At dimension 512, recursive Strassen multiplication with leaf size 128 took 7.0950 ms, while the matched OpenBLAS dense multiplication took 1.9597 ms.
- At dimension 2048, the tested recursive HODLR GPU path took 5.6079 ms, while dense GPU multiplication took 0.0860 ms.

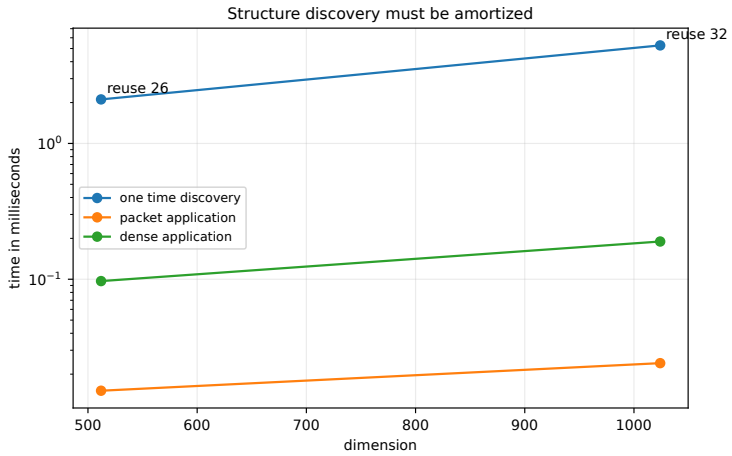


Figure 19. A profitable repeated representation can remain unprofitable for one use.

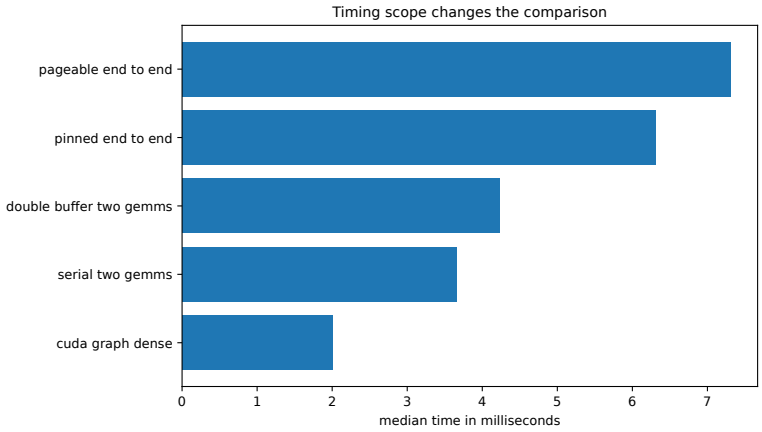


Figure 20. Device residence, transfer, graph replay, and a multioutput pipeline have different timing scopes.

- The tested FP32 iterative refinement solve took 22.0848 ms, while the direct FP64 GPU solve took 12.6286 ms with a comparable final error.
- The two stream dense path was slower than the serial two product path.
- The GPU Walsh Hadamard transform was slower than the CPU implementation at length 65536, then became faster at length 262144.

Fast multiplication can be asymptotically valuable and still lose at measured sizes because of recursion, memory traffic, and kernel granularity. Hierarchical matrix methods have similar crossover issues [1]. Transform algorithms and tensor factorizations form additional structure classes [7] and [23].

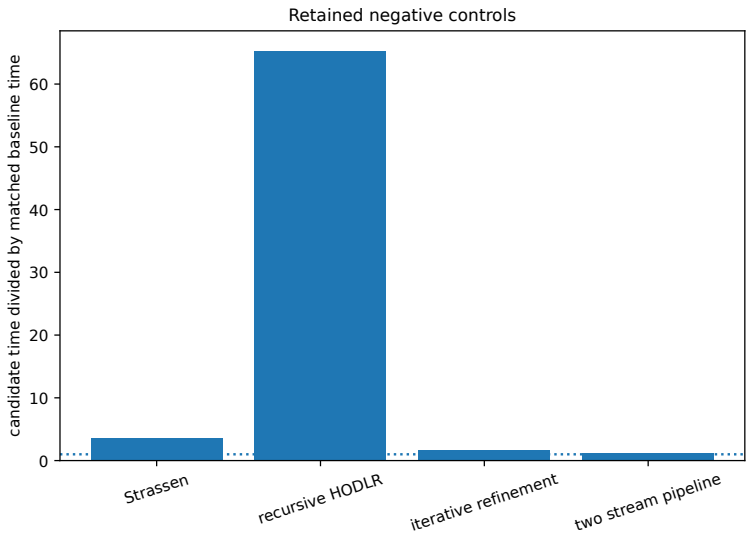


Figure 21. Values above one are slowdowns against the matched baseline.

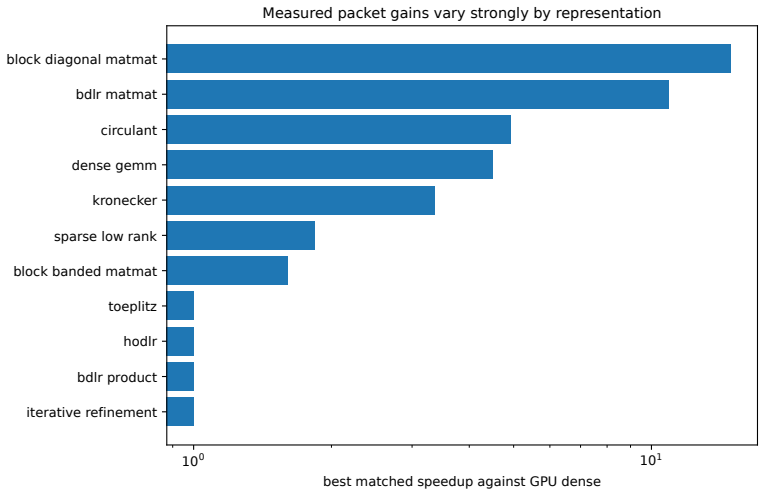


Figure 22. The best matched gain differs strongly across suites. Regular packet structure dominates the largest measured rows.

13. **Implementation and verification**

The release contains a small reference implementation of the exact operators and optimization algorithms used in the paper. The mathematical interface is deliberately narrow.

(1) A packet layout records dimensions and coordinate slices.

Layer	Verification target
Packet semantics	materialization, application, adjoint, composition
Low rank algebra	exact product, Woodbury solve, optimal core recompression
Graph algorithms	path support, topological levels, Schur fill inclusion
Optimization	Lagrangian tree compiler, budget compiler, precision assignment
Numerical bounds	product chain error, block perturbation, clipped allocation
Empirical corpus	row count, suite count, device split, status, error ceiling
Document	proofs, labels, citations, figures, font embedding, render audit

Table 3. Verification layers in the strengthened release.

(2) A packet operator stores active blocks and evaluates them without materializing inactive blocks.
(3) BDLR factors support exact application, product, solve, and small core recompression.
(4) Graph utilities compute support products, path closure, Schur fill inclusion, and topological levels.
(5) Tree dynamic programs solve the Lagrangian and quantized budget models.
(6) A multiple choice dynamic program assigns discrete precision options.
(7) Benchmark utilities audit case identity, output contracts, timing scopes, and retained negative controls.

The full deterministic verifier performs 37392 checks in nine groups. It covers packet evaluation, adjoints, composition, structured primitives, recompression optimality, product chain error, graph support, Schur fill, tree compilation, precision assignment, and the benchmark corpus. The strengthened unit suite contains eleven passing tests. The original source suite contains twenty three passing tests for BDLR operations, Krylov methods, mixed precision, measured data schema, and figure generation.

14. **Hardware implications and exact boundaries**

The finite results suggest a device interface with grouped block kernels, low rank contractions, packet metadata, a transfer graph scheduler, and per edge

precision tags. Such an interface may reduce materialization and repeated launch work when the operator family remains closed under the chosen representation.

The exact results stop at a finite candidate tree and a declared device model. Three broader problems remain open.

(1) A universal discovery algorithm would need to find a profitable packet structure from an arbitrary operator and data distribution. The simultaneous spectral theorem and randomized low rank approximation cover special cases only.
(2) A universal compiler would need a controlled approximation factor when costs are nonlocal and candidate decompositions overlap. The dynamic programs in theorems 9.4 and 9.8 do not cover that setting.
(3) A packet native memory hierarchy would need a communication theorem for a family of operators and a specified architecture. The one pass byte model in proposition 3.6 is not such a lower bound.

Communication lower bounds require explicit machine models [18] and [2]. CUDA Graph behavior and kernel constraints depend on the installed CUDA platform [22]. The measured results in this paper do not prove an optimal accelerator.

The article also makes no claim about the exponent of generic dense matrix multiplication. Packet algorithms answer a different question. They exploit a restricted operator class and may return a compressed output.

15. **Conclusion**

A packet operator representation records which block maps exist, how they are stored, and what output they denote. Exact block evaluation and composition turn the support graph into algebra. Arithmetic, storage, and one pass data movement follow from the active block dimensions. Block diagonal plus low rank operators remain closed under multiplication before recompression. Their inverse is reduced by Woodbury, and their correction can be recompressed optimally through a small singular value problem.

Sparse transfer graphs predict path support, triangular schedule depth, and possible fill under elimination. Packet diagonal structure gives local Krylov spaces and a certified block Jacobi condition bound. Packetwise perturbations give a direct mixed precision certificate. Continuous tolerances follow from a convex allocation, while finite data type choices lead to an exact multiple choice budget program.

A finite candidate refinement tree supports a genuine packet compiler. The Lagrangian optimum, the quantized error budget optimum, and local recompilation are exact under additive hypotheses. This is narrower than

unrestricted automatic discovery, but it turns one broad compiler conjecture into a proved algorithmic statement.

The measured corpus shows the practical distinction between regular and irregular structure. Equal block and BDLR operators gained substantially on the recorded GPU. Recursive HODLR, Strassen, iterative refinement, a small transform, and an attempted overlap lost to matched baselines. Discovery, transfer, capture, and materialization changed the break even point.

The computational principle is therefore precise. Remove work through a verified representation. Preserve the declared output. Compile only inside a model whose costs and errors are stated. Assign precision by sensitivity. Amortize one time costs. Keep negative controls. Treat the final timing as a property of the representation, device, kernel, and reuse regime together.

Language model contribution

An OpenAI language model was used to help reorganize the original manuscript, test alternative theorem formulations, draft portions of the exposition, and design finite verification calculations. Its mathematical suggestions affected the packet intermediate representation, the small core recompression theorem, the product chain error bound, the quantized precision program, the exact tree compiler, the local recompilation theorem, and the separation of benchmark contracts. The author is responsible for checking every statement, proof, citation, computation, and measured claim in any submitted version. The model is not an author.

References

1. Sivaram Ambikasaran and Eric Darve, *An o n log n fast direct solver for partial hierarchically semi separable matrices*, Journal of Scientific Computing **57** (2013), no. 3, 477–501.
2. Grey Ballard, James Demmel, Olga Holtz, and Oded Schwartz, *Minimizing communication in numerical linear algebra*, SIAM Journal on Matrix Analysis and Applications **32** (2011), no. 3, 866–901.
3. Richard Bellman, *Dynamic programming*, Princeton University Press, Princeton, NJ, 1957.
4. James R. Bunch and Christopher P. Nielsen, *Updating the singular value decomposition*, Numerische Mathematik **31** (1978), 111–129.
5. Erin Carson and Nicholas J. Higham, *Accelerating the solution of linear systems by iterative refinement in three precisions*, SIAM Journal on Scientific Computing **40** (2018), no. 2, A817–A847.
6. Ronald R. Coifman and Mladen Victor Wickerhauser, *Entropy based algorithms for best basis selection*, IEEE Transactions on Information Theory **38** (1992), no. 2, 713–718.

7. James W. Cooley and John W. Tukey, *An algorithm for the machine calculation of complex fourier series*, Mathematics of Computation **19** (1965), no. 90, 297–301.
8. Chandler Davis and William M. Kahan, *The rotation of eigenvectors by a perturbation. iii*, SIAM Journal on Numerical Analysis **7** (1970), no. 1, 1–46.
9. Timothy A. Davis, *Direct methods for sparse linear systems*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2006.
10. James Demmel, Ioana Dumitriu, and Olga Holtz, *Fast linear algebra is stable*, Numerische Mathematik **108** (2007), no. 1, 59–91.
11. Carl Eckart and Gale Young, *The approximation of one matrix by another of lower rank*, Psychometrika **1** (1936), no. 3, 211–218.
12. Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran, *Cache oblivious algorithms*, Proceedings of the Fortieth Annual Symposium on Foundations of Computer Science, IEEE Computer Society, 1999, pp. 285–297.
13. Gene H. Golub and William Kahan, *Calculating the singular values and pseudo inverse of a matrix*, Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis **2** (1965), no. 2, 205–224.
14. Gene H. Golub and Charles F. Van Loan, *Matrix computations*, fourth ed., Johns Hopkins University Press, Baltimore, MD, 2013.
15. William W. Hager, *Updating the inverse of a matrix*, SIAM Review **31** (1989), no. 2, 221–239.
16. Nathan Halko, Per Gunnar Martinsson, and Joel A. Tropp, *Finding structure with randomness*: *Probabilistic algorithms for constructing approximate matrix decompositions*, SIAM Review **53** (2011), no. 2, 217–288.
17. Nicholas J. Higham, *Accuracy and stability of numerical algorithms*, second ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, 2002.
18. Jia Wei Hong and H. T. Kung, *I o complexity*: *The red blue pebble game*, Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing, Association for Computing Machinery, 1981, pp. 326–333.
19. IEEE Standards Association, *Ieee standard for floating point arithmetic*, IEEE Std 754 2019, 2019.
20. Per Gunnar Martinsson and Joel A. Tropp, *Randomized numerical linear algebra*: *Foundations and algorithms*, Acta Numerica **29** (2020), 403–572.
21. Leon Mirsky, *Symmetric gauge functions and unitarily invariant norms*, The Quarterly Journal of Mathematics **11** (1960), 50–59.
22. NVIDIA Corporation, *Cuda programming guide*, NVIDIA Corporation, 2026, Version consulted May 2026.
23. Ivan V. Oseledets, *Tensor train decomposition*, SIAM Journal on Scientific Computing **33** (2011), no. 5, 2295–2317.
24. Yousef Saad, *Iterative methods for sparse linear systems*, second ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, 2003.
25. Volker Strassen, *Gaussian elimination is not optimal*, Numerische Mathematik **13** (1969), 354–356.

26. Samuel Williams, Andrew Waterman, and David Patterson, *Roofline*: *An insightful visual performance model for multicore architectures*, Communications of the ACM **52** (2009), no. 4, 65–76.

Komorebi P.

Paper VIII
Sixth Lean Verification Block

Small-Core Optimal Recompression
and Completion of the Finite Register

Komorebi P. September 6, 2026

Scope and result

Theorem 5.6, the last pending manuscript-proved item, is now covered. The finite register contains **56 of 56 checked items**. Four classical inputs remain separately tracked. This block adds **79 named theorem declarations in ten modules**, giving **392 declarations in 40 modules**. The total contains 373 Paper VIII declarations and 19 in the unchanged reused quantum-packet foundation. Definitions and generated declarations are not added to that theorem count.

The proof constructs compact orthonormal factors from the actual input column spaces. Given an *ordered thin singular-value decomposition of the actual small core*, it constructs thin square-root factors, proves their rank bound and global optimality in both the spectral and Frobenius norms, and proves the exact error formulas. The diagonal best-approximation inequalities are proved in this release, not assumed through an optimality certificate.

Fresh cumulative check	Result
Active Lean modules compiled	40/40
Named theorem dependency audits	392/392
Separately counted construction audits	66/66
Deliberately invalid Lean proof controls	Both rejected
Verifier, compatibility and sixth-block unit tests	43 passed
Independent numerical recompression cases	638 passed

Factorization boundary. As in the manuscript's instruction to take an SVD, its exact reconstruction, orthonormality, ordering and nonnegativity equations are supplied. No optimality or tail-error conclusion is supplied. General SVD existence, a floating-point SVD algorithm, and identification with the separately named Mathlib `singularValues` API are not independently formalized here. The exact column-space construction is not a numerical QR implementation.

All 30 upstream proof modules, the original manuscript assets and five earlier supplements remain byte-identical. The source archive is based on the 313-declaration fifth block, not the earlier same-named 298-declaration branch.

1. Compact factors from actual column spaces

Modules CompactRangeFactors and SmallCoreTransfer

Let

$$L \in \mathbb{C}^{m\times t}, \qquad R \in \mathbb{C}^{n\times t}, \qquad A = LR^*.$$

Use the ordinary Euclidean inner products on the coordinate spaces. Define

$$S_L = \operatorname{range}(L), \quad r_L = \dim S_L, \qquad S_R = \operatorname{range}(R), \quad r_R = \dim S_R.$$

The implementation constructs an orthonormal basis of each actual range subspace. Let Q_L and Q_R be the matrices of the corresponding isometric inclusions. Express the range-restricted input maps in these bases to obtain R_L, R_R.

The checked identities are

$$\boxed{Q_L^*Q_L = I, \quad L = Q_L R_L, \quad R_L = Q_L^* L,}$$

with the analogous identities on the right. The dimension theorem is not an input:

$$\boxed{r_L = \operatorname{rank} L, \qquad r_R = \operatorname{rank} R.}$$

The coordinate factors have full row rank. The construction therefore handles rank-deficient input factors without retaining artificial column directions.

Define the small core by actual multiplication,

$$G = R_L R_R^* \in \mathbb{C}^{r_L\times r_R}.$$

Finite matrix associativity and adjoint identities give

$$\boxed{A = Q_L G Q_R^*, \qquad \operatorname{rank} G = \operatorname{rank} A.}$$

The rank identity follows from both multiplication inequalities and the left inverse obtained by compression. It is not inferred from agreeing dimensions.

Zero factors and empty spaces. No positive-rank or nonempty-coordinate hypothesis is imposed. If a range is zero, its orthonormal basis has an empty index type. The factorization uses the resulting rectangular empty matrices. A separate theorem proves that the compact dimension is zero exactly when the original factor is zero.

Relation to QR. The argument only needs orthonormal columns and exact reconstruction. A compact QR factorization supplies these properties, but triangularity of its coordinate factor is irrelevant to the recompression theorem. The formal construction uses an exact orthonormal range basis; it does not claim Householder steps, pivot selection or floating-point stability. A supplied noncompact orthonormal factorization is also covered by the general transfer theorem, including extra zero singular directions.

2. Why a small-core optimum is a global optimum

Module SmallCoreNorms; arbitrary ambient competitors

For rectangular isometries Q, V define

$$\mathcal{L}(G) = QGV^*, \qquad \mathcal{C}(B) = Q^*BV, \qquad Q^*Q = I, \quad V^*V = I.$$

The following statements are proved for the actual matrix rank and actual matrix norms:

$$\mathcal{C}(\mathcal{L}(G)) = G, \qquad \operatorname{rank}\mathcal{L}(G) = \operatorname{rank} G, \qquad \operatorname{rank}\mathcal{C}(B) \le \operatorname{rank} B,$$

$$\boxed{\|\mathcal{L}(G)\|_\nu = \|G\|_\nu, \qquad \|\mathcal{C}(B)\|_\nu \le \|B\|_\nu, \quad \nu \in \{2, \mathrm{F}\}.}$$

For the spectral norm, the proof uses the Gram matrix identity

$$(QG)^*(QG) = G^*G$$

and the C^*-norm square identity, together with right-side adjoint symmetry. Compression follows from submultiplicativity and the isometry norm bound, including empty spaces.

For the Frobenius norm, trace cyclicity proves left and right isometric invariance. The proof also establishes the projection identity

$$\|B - QQ^*B\|_{\mathrm{F}}^2 = \|B\|_{\mathrm{F}}^2 - \|Q^*B\|_{\mathrm{F}}^2.$$

Nonnegativity of the left side gives left compression contractivity; the right version follows by adjoints. No norm-preservation or contractivity hypothesis is inserted.

Suppose G_k is a best rank-at-most-k approximation to G in either norm. Take an *arbitrary* ambient matrix B with $\operatorname{rank} B \le k$. Compression is an admissible small-core competitor and

$$\begin{aligned} \|\mathcal{L}(G) - \mathcal{L}(G_k)\|_\nu &= \|G - G_k\|_\nu \\ &\le \|G - \mathcal{C}(B)\|_\nu \\ &= \|\mathcal{C}(\mathcal{L}(G) - B)\|_\nu \\ &\le \|\mathcal{L}(G) - B\|_\nu. \end{aligned}$$

This is the key global bridge: B is not required to lie in the supplied row and column spaces. The next two pages discharge the intermediate small-core optimality obligation, rather than leaving it as a hypothesis in the final theorem.

3. Spectral optimality from a kernel witness

Modules DiagonalTruncation and SpectralOptimality

Let $\sigma_1 \geq \cdots \geq \sigma_d \geq 0$ and

$$D = \operatorname{diag}(\sigma_1, \ldots, \sigma_d), \qquad D_k = \operatorname{diag}(\sigma_1, \ldots, \sigma_{\min(k,d)}, 0, \ldots, 0).$$

Lean uses indices beginning at zero; the formulas on this page use the conventional one-based notation. The rank of D_k is proved to be at most k, even when retained singular values are zero.

For $k < d$, take any $d \times d$ matrix B of rank at most k. Restrict its domain to the first $k+1$ coordinates using the actual coordinate inclusion J. Since

$$\operatorname{rank}(BJ) \leq \operatorname{rank} B \leq k < k+1,$$

rank-nullity supplies a nonzero vector z with $BJz = 0$. The vector $y = Jz$ is nonzero, satisfies $By = 0$ and is supported in the first $k+1$ coordinates. The injection and the support property are checked separately.

Monotonicity of the diagonal entries gives

$$\|Dy\|^2 = \sum_{i=1}^{k+1} \sigma_i^2 |y_i|^2 \geq \sigma_{k+1}^2 \|y\|^2 .$$

Because $(D - B)y = Dy$, the operator-norm inequality implies

$$\sigma_{k+1} \|y\| \leq \|(D - B)y\| \leq \|D - B\|_2 \|y\| .$$

Cancellation of the strictly positive norm of y proves

$$\boxed{\sigma_{k+1} \leq \|D - B\|_2 .}$$

The norm of the actual diagonal residual $D - D_k$ equals σ_{k+1}, so D_k attains the lower bound. If $k \geq d$, the residual is zero and optimality follows from norm nonnegativity.

No optimality oracle. The nonzero kernel vector, its coordinate support and the diagonal norm bound are all derived. No Eckart–Young inequality, best-rank property or singular-tail formula is assumed. The final rectangular SVD result follows by the isometric transfer proved on the preceding page.

4. Frobenius optimality from kernel weights

Modules KernelFrame and FrobeniusOptimality

For an arbitrary competitor B with $\operatorname{rank} B \le k$, construct an orthonormal matrix Q spanning its actual kernel. Then

$$BQ = 0, \qquad Q^*Q = I, \qquad q = \dim\ker B = d - \operatorname{rank} B.$$

Define the row weights

$$w_i = \sum_{j=1}^{q} |Q_{ij}|^2.$$

The proof derives

$$\boxed{0 \le w_i \le 1, \qquad \sum_i w_i = q \ge \max(d-k, 0).}$$

The upper bound uses contraction of Q^* on a coordinate vector. The weight sum follows from the trace of Q^*Q and rank-nullity.

For any nonnegative decreasing sequence a_i, weights with these properties satisfy

$$\sum_i a_i w_i \ge \sum_{i>k} a_i.$$

This finite weighted inequality is itself proved. For $k < d$, use $\mu = a_{k+1}$ and compare the summands to tail weights $v_i = 0$ for $i \le k$ and $v_i = 1$ otherwise:

$$a_i v_i + \mu(w_i - v_i) \le a_i w_i.$$

For a retained coordinate the inequality uses $a_i \ge \mu$ and $w_i \ge 0$. For a tail coordinate it uses $a_i \le \mu$ and $w_i \le 1$. Summation and $\sum w_i \ge \sum v_i$ give the result.

Set $a_i = \sigma_i^2$. Right compression cannot increase the Frobenius norm, and $BQ = 0$, hence

$$\begin{aligned} \|D - B\|_{\mathrm{F}}^2 &\ge \|(D-B)Q\|_{\mathrm{F}}^2 \\ &= \|DQ\|_{\mathrm{F}}^2 = \sum_i \sigma_i^2 w_i \ge \sum_{i>k} \sigma_i^2. \end{aligned}$$

Direct calculation of $D - D_k$ attains equality. Thus the best-rank Frobenius assertion is proved from kernel geometry and finite sums, independently of the spectral lower-bound proof. Repeated and zero singular values require no uniqueness claim.

5. Thin output factors and the final theorem

Modules CompactTruncation and SmallCoreRecompression

For the actual core $G = R_L R_R^*$, supply ordered thin SVD data

$$G = W \operatorname{diag}(\sigma) Z^*, \qquad W^*W = I, \qquad Z^*Z = I, \qquad \sigma_1 \geq \cdots \geq \sigma_d \geq 0.$$

These are precisely the fields of the formal SVD structure. It contains no rank-optimality or tail-error field. A derived theorem checks the Gram eigenvector equation

$$G^*GZ = Z \operatorname{diag}(\sigma_1^2, \ldots, \sigma_d^2).$$

Let $h = \min(d, k)$ and retain exactly the first h column indices. The rectangular square-root mask has entries

$$M_{ij} = \begin{cases} \sqrt{\sigma_j}, & i = j, \\ 0, & i \neq j, \end{cases} \qquad 1 \leq i \leq d, \quad 1 \leq j \leq h.$$

Lean proves its index cardinality and $MM^* = D_k$. Define

$$\boxed{L_k = Q_L W M, \qquad R_k = Q_R Z M.}$$

The product equals $Q_L W D_k Z^* Q_R^*$ and has rank at most k. The principal theorem proves, for every ambient B with $\operatorname{rank} B \leq k$,

$$\boxed{\|LR^* - L_k R_k^*\|_\nu \leq \|LR^* - B\|_\nu, \quad \nu \in \{2, \mathrm{F}\}.}$$

It also gives the exact errors

$$\|LR^* - L_k R_k^*\|_2 = \begin{cases} \sigma_{k+1}, & k < d, \\ 0, & k \geq d, \end{cases} \qquad \|LR^* - L_k R_k^*\|_{\mathrm{F}} = \left(\sum_{i>k} \sigma_i^2 \right)^{1/2}.$$

Both the squared and square-root Frobenius formulas are checked. All these statements are combined in

`KomorebiPaperVIII.SmallCore.small_core_optimal_recompression`.

Endpoints. Rank zero produces the zero output; $k \geq d$ reconstructs the original product. An empty SVD certificate is constructed for a zero core, and diagonal SVD certificates are constructed explicitly. Zero singular values are never inverted. At repeated singular values the result asserts optimality, not uniqueness of the minimizer.

6. Source map and verification

One completed manuscript item; 79 supporting declarations

Module	Main purpose	Theorems
SmallCoreNorms	Isometric norms, compression, rank	19
CompactRangeFactors	Actual compact column factors	7
SmallCoreTransfer	Global best-rank transfer	7
KernelFrame	Actual kernel and row weights	8
DiagonalTruncation	Rank and exact diagonal errors	9
FrobeniusOptimality	Weighted-tail lower bound	5
SpectralOptimality	Prefix-kernel lower bound	7
CompactTruncation	Thin square-root factors	5
SmallCoreRecompression	Main theorem and Gram relation	6
RecompressionEndpoints	Zero and full-retention cases	6
Total	Theorem 5.6 and its support	79

Cumulative Lean run. All 40 local modules were compiled in a newly created output directory under Lean 4.34.0-rc2. Each exited with code zero and warnings were fatal. The 392 named theorem reports and 66 separately counted construction reports match the inventory exactly. Dependencies are confined to `propext`, `Classical.choice` and `Quot.sound`, or subsets thereof. False equality and admitted-proof controls were both rejected.

Pinned dependency. The actual Mathlib Git revision and manifest revision both match

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`. The supplied compiled Mathlib cache was reused. It was not independently rebuilt or rechecked. Recovery verified all seven offline archive parts and the combined archive hash.

Infrastructure. Forty-three unit tests passed: 17 verifier tests, 12 fifth-block compatibility tests updated to the current totals, and 14 sixth-block tests. The original fifth-block fixed-count test file and metadata are preserved. Exact audit coverage, dependency order, no-bypass scans, classical-input boundaries and absence of optimality fields in the SVD structure are tested.

Failures retained. Of 31 captured development checks, 21 failed before correction and ten accepted drafts supplied the new modules. Failures concerned proof elaboration, finite-index/cardinality APIs, dependent rewrites, norm scopes and warning diagnostics. Their inactive source snapshots and logs are retained separately from the successful fresh cumulative run. An initial interactive runtime launch was unsupported; noninteractive assembly succeeded.

7. Regressions, reproduction and claim boundary

Exact mathematical guarantees versus numerical implementations

Independent numerical checks. A deterministic NumPy complex128 mirror tested 182 input-factor pairs at 638 truncation cases. The cases include rectangular matrices, rank-deficient and zero factors, empty spaces, repeated singular values, rank zero and ranks beyond the core width. Comparison used both the small core and an independently computed ambient SVD reference. There were 3,190 arbitrary rank-bounded competitors, nine invalid-input rejection controls and two checks demonstrating failure when essential assumptions are removed.

The largest relative tail/reconstruction discrepancy was below 3.7×10^{-15}, against a declared 2×10^{-9} tolerance. The mirror uses numerical rank thresholds and a numerical SVD to construct column bases. It is not compiled or extracted Lean code, a proof for all inputs, or a GPU benchmark. The formal result uses exact ranks and exact factorization equations.

Reproduce from the extracted source root.

```
python3 scripts/verify_integrity.py
python3 -m unittest discover -s scripts -p 'test_*.py' -v
python3 scripts/verify.py --kit /path/to/portable-kit \
    --out verification/recheck --timeout 240
python3 scripts/run_sixth_regressions.py \
    --out verification/recheck_numerical.json
```

The unit tests and numerical mirror require NumPy. The Lean verification path needs Python 3.10 or later and the separately supplied pinned Linux x86-64 kit. Local compiled interfaces are regenerated. The runner also supports bounded batches with `-max-checks` and hash-checked `-resume`. A partial checkpoint is not a pass.

What is complete. The 56-item manuscript-proved register is covered under the recorded hypotheses. The original theorem says to take orthonormal factors and a core SVD. The new construction and proof discharge the recompression conclusion from those exact factorization equations. In particular, no best-approximation assertion or error formula is hidden in an input certificate.

What is not claimed. General SVD existence or a numerical SVD implementation is not newly formalized. The four classical entries remain separately tracked; the ordered-SVD-data optimality argument now has checked support, while the CG algorithm theorem, floating-point product model and simultaneous-decomposition input are not newly proved in full. No new generic matrix-multiplication exponent, floating-point stability theorem or hardware speedup follows from this release.

The original manuscript, bibliography, source package and all five earlier supplements are preserved. Historical CPU/GPU measurements, earlier compiler execution suites and Papers I–VII in full were not rerun. Prose-to-formal correspondence is an explicit mathematical review, not an automated theorem equating a sentence with a Lean type.

THREE VIEWS OF ONE COMPUTATION

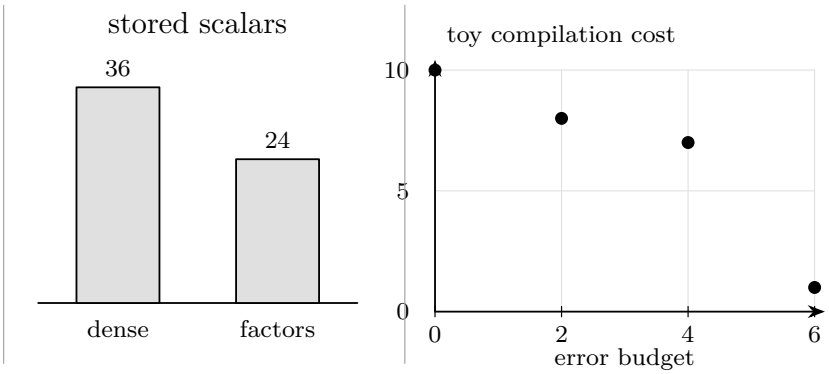
Compute the representation first

Reading source: main text, Sections 2–3; sixth-block supplement, p. 1.

A dense array, a factored operator, and a compiled execution plan are different objects. They may denote the same linear map, but they need not store the same data, return the same output, or incur the same costs. This appendix makes those distinctions visible through calculations rather than a pipeline of boxes.

A: 36 visible entries

4	2	1	1	1	1
2	4	1	1	1	1
1	1	4	2	1	1
1	1	2	4	1	1
1	1	1	1	4	2
1	1	1	1	2	4



Three models to carry through the paper

The first is a six-dimensional operator $A = D + uv^T$. Every entry of A is nonzero, yet three 2×2 diagonal blocks and two thin vectors suffice to apply it exactly. Pages V – 3, V – 9, V – 10, and V – 27 reuse this operator or its factors.

The second starts with redundant factors L and R. Actual range coordinates reduce a 4×3 product to a 2×3 core. Pages V – 11–V – 12 connect rank, singular tails, and the geometric reason a small-core optimum is also an ambient optimum.

The third is a seven-node candidate tree with exactly five prunings. Its budget optimum is not selected by any Lagrangian penalty. Pages V – 21–V – 23 let readers inspect every feasible choice and every changed subproblem.

How to use the appendix

A source line beneath each title links to the relevant preserved manuscript or supplement page in the combined edition. Exact checks use rational, Gaussian-rational, symbolic, and finite-enumeration arithmetic. Smooth drawing coordinates are illustrations of stated formulas, not additional proof evidence.

> **Read this as:** An exact representation can remove work before a device is chosen. It does not by itself supply a fast kernel, a good approximation, or a fair timing comparison.

A READER'S ATLAS

The objects and their contracts

Main text: Sections 2–3, 8–9, and 11.

The same letter can refer to a mathematical operator or to a particular stored representation. Keep those levels separate when reading an algorithm or a benchmark row.

Symbol	Object	Question to ask
P_α	Coordinate projector	Which input or output coordinates does it select?
$A_{\beta\alpha}$	A block map	What are its source and target dimensions?
E	Active block addresses	Is this exact support, or only a structural upper bound?
$D + UV^*$	Factored operator	Is the desired output a vector, factors, or a dense array?
Q_L, G, Q_R	Range factors and core	Are the outer columns orthonormal, with exact reconstruction?
E_j	An error operator	Is it a factor perturbation or an additive error after a product?
s_e, ϵ_e	Sensitivity and tolerance	Is the budget input-dependent or uniform over unit inputs?
$(\kappa_v, \epsilon_v, \sigma_v)$	Candidate-tree data	Do costs, errors, and split overheads really add?
$F_v(b)$	A budget table	Does b mean exact weight, or an upper bound?
R	Reuse count	Which one-time costs have been included?

Conventions used in the examples

Matrices act on column vectors. A block $A_{\beta\alpha}$ goes from source α to target β. Products act from right to left. Real examples use transpose; complex examples use adjoint unless a vectorization identity explicitly requires transpose. Column-major vectorization stacks the columns of a matrix.

An arithmetic count follows the paper's modeled multiply/add convention. It is not a count of executed machine instructions. The tiny scheduling and amortization examples use declared synthetic units, not measured GPU times. The original benchmark corpus remains historical data in the preserved source tree.

Evidence levels

The original theorem ledger records its inherited finite formal scope. The new exact checks validate these particular examples. A plot samples a formula. A historical timing describes its recorded case and machine. None of these evidence types can silently substitute for another.

> **Read this as:** Before interpreting a number, identify its object, norm, output contract, and timing scope. Most apparent contradictions disappear once those four fields are explicit.

THE RUNNING SIX-DIMENSIONAL OPERATOR

A dense-looking matrix, one scalar coupling

Main text: Theorems 2.3 and 3.4, pp. 4 and 7.

Let D have three identical blocks $\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ and put $u = v = \mathbf{1} \in \mathbb{Q}^6$. Then $A = D + uv^T$ has diagonal entries four, within-block off-diagonal entries two, and every cross-block entry one. All 36 entries are nonzero.

$$x = (1, -1, 2, 0, -2, 3)^T, \qquad v^T x = 3, \qquad Ax = Dx + 3u.$$

x		Dx		$u(v^T x)$		Ax
1		2		3		5
−1		−2		3		1
2		6		3		9
0	+	2	+	3	=	5
−2		−3		3		0
3		7		3		10

$v^T x = 3$ is shared by all six output coordinates.

What was avoided?

The global part does not require 36 independent coefficients. It depends on the input through one scalar, $v^T x$, which is broadcast through u. The diagonal part is computed by three separate local matrix products. Nothing is approximated and no channel is discarded.

The sixth output is $7 + 3 = 10$, while the fifth is $-3 + 3 = 0$. A zero result can arise through cancellation even when the corresponding row contains no zero coefficient. The output vector is therefore not a support map of the input matrix.

What has not been established?

This computation proves neither that the supplied factors are the cheapest representation nor that six small tasks outperform one dense kernel. For example, the repeated diagonal blocks and all-one vectors admit further special handling. The stated BDLR model deliberately treats the blocks as dense and the factors as stored vectors so that the paper's general formulas apply literally.

The checker constructs the complete matrix independently, multiplies it by x, and compares all six entries with the factored route. The same input returns in the product calculation on V – 27.

> **Read this as:** A matrix can have full entrywise support and still admit a small exact representation. Entry sparsity and algebraic compressibility are different properties.

ARITHMETIC, STORAGE, AND CAPACITY

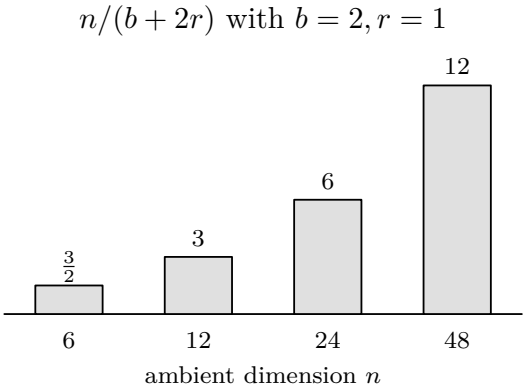
Work removed is not time measured

Main text: Theorems 3.1 and 3.4; Corollaries 3.2 and 3.9, pp. 7–9.

For q right-hand sides, the paper models dense application by $2n^2q$ operations. A block-diagonal-plus-rank-r representation with equal block width b uses

$$F = 2qn(b + 2r), \qquad M = n(b + 2r) \text{ stored scalars.}$$

These formulas count the dense block products and two thin products. They follow the manuscript convention rather than adding every scalar accumulation or index operation.



Six-dimensional running operator

dense: 36 scalars, $72q$ work

BDLR: 24 scalars, $48q$ work

ratio: $3/2$, not a timing result

For the running example, $n = 6, b = 2, r = 1$. The model reduces storage from 36 to 24 scalars and application work from $72q$ to $48q$. For fixed b, r, the ratio $n/(b + 2r)$ grows with n. If the rank instead grows proportionally with dimension, that apparent advantage need not grow.

Memory is another constraint

With scalar size s, the representation alone requires sM bytes. Inputs, outputs, workspace, allocator overhead, and stored indices are additional. A matrix-only capacity inequality is not a complete device-memory fit test.

For a block band of p packets and half-bandwidth w, the exact active-block count is

$$p(2w + 1) - w(w + 1).$$

The subtraction removes overcounted edge diagonals. The checker enumerates all addresses for $1 \le p \le 6$ and every admissible w, including $w = 0$ and $w = p - 1$.

> **Read this as:** An arithmetic or storage ratio explains a representation advantage. Kernel launch cost, traffic, utilization, and conversion determine whether that advantage becomes a time advantage.

CIRCULANT MULTIPLICATION

Four coefficients, four exact frequencies

Main text: Theorem 4.3, p. 11.

Take $c = (2, 1, 0, -1)^T$ and define $C(c)_{jk} = c_{j-k \bmod 4}$. The unitary Fourier matrix is $F_{jk} = \mathrm{i}^{-jk}/2$. Its normalization is visible in the exact diagonalization:

$$C(c) = F^* \operatorname{diag}(2Fc) F, \qquad 2Fc = (2, 2-2\mathrm{i}, 2, 2+2\mathrm{i})^T.$$

$C(c)$

2	−1	0	1
1	2	−1	0
0	1	2	−1
−1	0	1	2

$\sqrt{4}\,Fc$

2
$2-2i$
2
$2+2i$

$$F_{jk} = \tfrac{1}{2}\,\mathrm{i}^{-jk}$$

$$C = F^* \operatorname{diag}(\sqrt{4}Fc) F$$

$\sqrt{4} = 2$ is essential.

A unitary transform changes the normalization of the convolution eigenvalues.

Follow an actual input

For $x = (1, 2, -1, 0)^T$, dense multiplication gives $(0, 6, 0, -2)^T$. Transforming x, multiplying by the four displayed eigenvalues, and applying F^* gives exactly the same vector. Every entry belongs to $\mathbb{Q}(\mathrm{i})$, so the check needs no approximate trigonometric values.

There is a common normalization trap. With an unnormalized discrete transform, the convolution multiplier has one convention. With the unitary transform used here, the eigenvalue vector is $\sqrt{n}\,Fc$, not Fc. Dropping the factor two reconstructs $C/2$ rather than C in this example. The invalid control deliberately tries that change and rejects it.

Where the fast algorithm enters

The identity specifies the represented operator. An FFT is an additional factorization that evaluates the transforms efficiently. The algebraic diagonalization does not certify a particular floating-point FFT, data layout, or launch policy. For a tiny four-point example, direct multiplication may be the simpler execution path even though the frequency representation is exact.

> **Read this as:** Transform coordinates change both the storage pattern and the normalization conventions. Verify the inverse transform and convolution scale before interpreting a frequency-domain product.

TOEPLITZ EMBEDDING WITHOUT WRAPAROUND

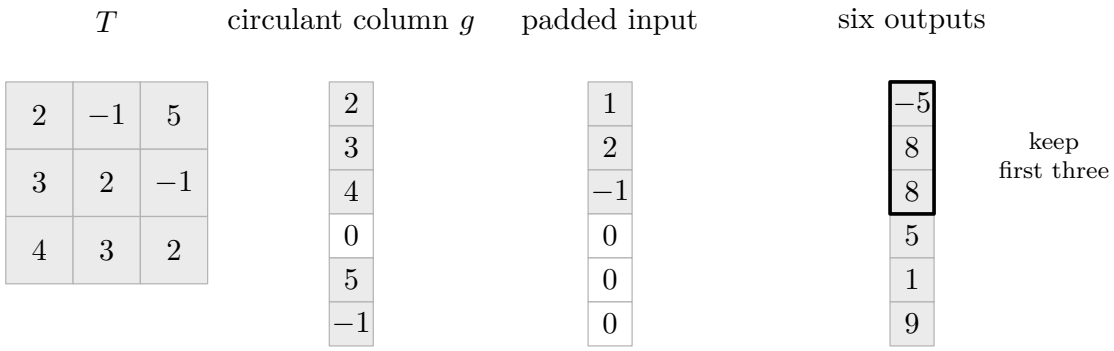
Padding is part of the theorem

Main text: Corollary 4.4, p. 12.

Consider

$$T = \begin{pmatrix} 2 & -1 & 5 \\ 3 & 2 & -1 \\ 4 & 3 & 2 \end{pmatrix}, \qquad x = (1, 2, -1)^T.$$

The diagonal pattern is Toeplitz, not circulant. To use a circular convolution, embed it into dimension six with first column $g = (2, 3, 4, 0, 5, -1)^T$ and pad the input with three zeros.



Padding is part of the operator identity, not optional workspace.

Read the wrapped indices

For output index $j < 3$, an input index $k \le j$ uses the first-column coefficient c_{j-k}. An input index $k > j$ wraps to the stored first-row coefficient r_{k-j}. The padded input positions contribute nothing. Keeping the first three outputs therefore gives

$$Tx = (-5, 8, 8)^T.$$

The remaining circular outputs are workspace consequences of the embedding, not additional components of Tx.

A concrete failure mode

Using only the first column $(2, 3, 4)$ as a three-dimensional circulant changes the upper-triangular coefficients. That is a different matrix. The checker compares its output with Tx and rejects the unpadded shortcut.

The middle zero in g is not a discarded Toeplitz coefficient: it separates the lower and reversed upper diagonals in the chosen $2n$ embedding. Other embeddings require their own index identities. It is not sufficient for an array merely to have the right length or to produce a plausible plot.

> **Read this as:** Zero padding prevents the wrong circular interactions. It is an algebraic part of the operator representation, not an optional performance tweak.

KRONECKER ACTION AND ARRAY ORDER

A transpose, not an adjoint

Main text: Theorem 4.5 and Corollary 4.6, p. 12.

With column-major vectorization, the tensor identity is

$$(A \otimes B)\,\mathrm{vec}(X) = \mathrm{vec}(BXA^T).$$

Here the transpose records index order, not a Hermitian inner product. Use the explicitly complex matrix $A = \begin{pmatrix} 1 & \mathrm{i} \\ 2 & -1 \end{pmatrix}$, together with $B = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$.

X

1	2
3	4

$\mathrm{vec}(X)$

1
3
2
4

BXA^T

$5 + 8i$	2
$9 + 12i$	6

A^T
not A^*

$$(A \otimes B)\,\mathrm{vec}(X) = \mathrm{vec}(BXA^T)$$

Check the coordinates

The stacked input is $(1, 3, 2, 4)^T$. Direct tensor multiplication agrees with stacking the columns of

$$BXA^T = \begin{pmatrix} 5 + 8\mathrm{i} & 2 \\ 9 + 12\mathrm{i} & 6 \end{pmatrix}.$$

Replacing A^T with A^* changes the signs of the imaginary terms. A real-only test would conceal that error, which is why the worked model uses an imaginary coefficient.

Why modewise contraction matters

For A of order m and B of order n, the explicit tensor matrix stores m^2n^2 entries. Its factors store $m^2 + n^2$. Applying through BXA^T costs $2n^2m + 2nm^2$ modeled operations, rather than forming the full tensor matrix first.

A different flattening convention has a different coordinate formula. The statement is not independent of how an array is reshaped. In a software implementation, shape, stride, and flattening order belong to the correctness contract as well as the cost model.

> **Read this as:** Tensor structure gives exact savings only when the address convention is preserved. A silent transpose, conjugation, or reshape change implements a different map.

WALSH RECURSION IN EIGHT COORDINATES

See every butterfly value

Main text: Theorem 4.7, p. 13.

The unnormalized Walsh matrix satisfies $H_1 = [1]$ and

$$H_{2n} = \begin{pmatrix} H_n & H_n \\ H_n & -H_n \end{pmatrix}, \qquad H_n H_n^T = nI.$$

Apply the three add/subtract stages to $(1, 2, 3, 4, 5, 6, 7, 8)^T$. The full intermediate ledger is displayed below.

input	1	2	3	4	5	6	7	8
stage 1	3	-1	7	-1	11	-1	15	-1
stage 2	10	-2	-4	0	26	-2	-4	0
stage 3	36	-4	-8	0	-16	0	0	0

$8 \log_2 8 = 24$ additions/subtractions; $H_8 H_8^T = 8I$.

Arithmetic can be counted exactly

Each stage visits eight coordinates, producing one sum or difference per coordinate. Three stages therefore use exactly 24 additions or subtractions. No multiplication by a dense 8×8 matrix is necessary.

The result $(36, -4, -8, 0, -16, 0, 0, 0)^T$ still reconstructs the input: the inverse is $H_8^T/8$. Because the displayed transform is unnormalized, its squared output norm is eight times the squared input norm. The normalized transform $H_8/\sqrt{8}$ would instead be orthogonal.

What the zeros mean

The zero coefficients arise from exact cancellation among the input values. They do not imply that the transform's defining matrix is sparse. The butterfly structure, not the final coefficient support, explains the arithmetic reduction.

The checker verifies the recursion, orthogonality, dense-versus-butterfly output, and operation count for sizes one, two, four, eight, and sixteen. These bounded tests exercise the implementation and boundary cases. The general recursion theorem remains the manuscript's mathematical statement.

> **Read this as:** Factorization can turn a dense matrix into a short exact arithmetic circuit. Keep its normalization separate from its recursive evaluation rule.

CLOSED LOW-RANK PRODUCT ALGEBRA

The cross term has somewhere to go

Main text: Theorem 5.1 and Corollary 5.2, pp. 13–14.

Let $A = D_A + u_a v_a^T$ be the running operator. Take $D_B = \mathrm{diag}(1,2,3,4,5,6)$, $u_b = (1,0,1,0,1,0)^T$, and $v_b = (0,1,0,1,0,1)^T$. Define $B = D_B + u_b v_b^T$.

$$U_C = [D_A u_b + u_a(v_a^T u_b) \;\; u_a], \qquad V_C = [v_b \;\; D_B^T v_a].$$

U_C

6	1
4	1
6	1
4	1
6	1
4	1

V_C

0	1
1	2
0	3
1	4
0	5
1	6

$$AB = D_A D_B + U_C V_C^T$$

$$U_A(V_A^T U_B)V_B^T$$

must be retained

Two input rank-one corrections produce at most rank two, not three.

Why there are only two columns

Expanding AB gives three correction terms, but two share the right factor v_b^T:

$$D_A u_b v_b^T + u_a(v_a^T u_b)v_b^T = (D_A u_b + u_a(v_a^T u_b))v_b^T.$$

The remaining term is $u_a v_a^T D_B$. Thus $AB = D_A D_B + U_C V_C^T$ with at most two correction directions. Counting the three expanded summands as independent ranks would overestimate this representation.

The scalar $v_a^T u_b$ is three. Removing the quadratic term would therefore remove $3u_a v_b^T$, which is nonzero. The checker compares the complete product to the factor construction and rejects a calculation that omits this contribution.

Before and after recompression

A rank bound on stored factors is not always the actual matrix rank. Repeated products may introduce dependent columns. Recompression can remove those dependencies exactly, or remove small singular directions approximately under an explicit error budget.

> **Read this as:** Closure of a representation class is an algebraic statement about all product terms. It is not permission to drop an interaction merely because it is second order in the factors.

WOODBURY WITH BOTH INVERTIBILITY CHECKS

A large inverse, a small denominator

Main text: Theorem 5.3 and Corollary 5.4, pp. 14–15.

For the running operator, each diagonal block has inverse $\frac{1}{8}\begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$. Hence $D^{-1}u = u/4$ and

$$S = 1 + u^T D^{-1} u = \frac{5}{2}.$$

The Woodbury identity reduces the entire inverse to

$$A^{-1} = D^{-1} - D^{-1} u S^{-1} u^T D^{-1} = D^{-1} - \frac{1}{40} u u^T.$$

one block of D^{-1}

$\frac{3}{8}$	$-\frac{1}{8}$
$-\frac{1}{8}$	$\frac{3}{8}$

$$D^{-1}u = \tfrac{1}{4}u$$

$$S = 1 + u^T D^{-1} u = \tfrac{5}{2}$$

$$A^{-1} = D^{-1} - \tfrac{1}{40} u u^T$$

$I - e_1 e_1^T$

0	0
0	1

invertible D
singular update

Two hypotheses, not one

Invertibility of D alone does not suffice. For $D = I_2$, $u = e_1$, and $v = -e_1$, the update is $\mathrm{diag}(0, 1)$. The small matrix S is zero, exactly where the correction formula would require an invalid division.

For the running model, the checker multiplies the proposed inverse by A in both orders and obtains the identity. It also checks the determinant relation $\det A = \det D \det S$. These tests verify the actual factors and sign of the correction, not just a numerical solve residual.

Conditioning survives the reduction

A small S can still be ill-conditioned. Reducing its dimension does not bound $\|S^{-1}\|$. The manuscript's inverse estimate keeps that term explicit, along with the norms of $D^{-1}u$ and $v^T D^{-1}$.

> **Read this as:** The hidden coupling has not vanished. Its solvability and sensitivity are concentrated in a smaller operator that still has to be invertible and numerically controlled.

ACTUAL RANGE COORDINATES

Stored width is not intrinsic rank

Main text: Theorem 5.6, p. 15; sixth-block supplement, p. 2.

Choose the isometry

$$Q_L = \begin{pmatrix} 3/5 & 0 \\ 4/5 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad R_L = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad R_R = \begin{pmatrix} 3 & 0 & 2 \\ 0 & 2 & 0 \\ 1 & 0 & -1 \end{pmatrix}.$$

Set $L = Q_L R_L$ and $R = R_R$. The two equal columns of L do not create a third range direction. In contrast, R has rank three.

L: rank 2

$\frac{3}{5}$	0	$\frac{3}{5}$
$\frac{4}{5}$	0	$\frac{4}{5}$
0	1	0
0	0	0

R: rank 3

3	0	2
0	2	0
1	0	-1

G: 2×3

5	0	0
0	2	0

$$LR^T = Q_L G$$

The stored width is three; the left range has only two independent directions.

The actual core is

$$G = R_L R_R^T = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 2 & 0 \end{pmatrix}, \qquad LR^T = Q_L G.$$

Its dimensions are the two actual input ranks, two and three. This is not inferred by merely discarding a column with a small norm. The range factor Q_L has orthonormal columns and reconstructs L exactly.

The preserved formal boundary

The sixth release constructs orthonormal bases of the actual column spaces. It then assumes exact ordered thin-SVD data for the core, as the manuscript instructs the reader to take an SVD. It does not assume an optimality or tail-error conclusion. Nor does it certify a floating-point QR or SVD implementation simply because those routines use similar notation.

This example has a diagonal rectangular core, so its singular directions are explicit. No iterative numerical factorization is needed to identify the two nonzero singular values, five and two.

> **Read this as:** Compactness is a statement about actual range spaces. The number of stored factor columns can exceed the dimensions that matter for recompression.

COMPRESSION, KERNEL WITNESSES, AND TAILS

Why a core optimum is global

Reading source: sixth-block supplement, pp. 3–6.

The product from the previous page is

$$A_0 = LR^T = \begin{pmatrix} 3 & 0 & 0 \\ 4 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Keeping only its first singular direction gives a rank-one approximation with spectral and Frobenius error exactly two. Keeping none gives errors five and $\sqrt{29}$; keeping both gives zero.

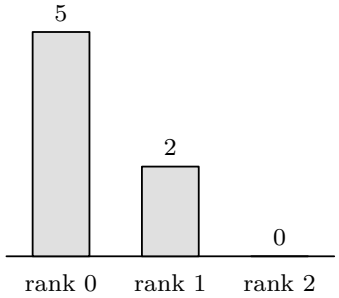
LR^T

3	0	0
4	0	0
0	2	0
0	0	0

rank-one output

3	0	0
4	0	0
0	0	0
0	0	0

spectral error



Frobenius errors: $\sqrt{29},\ 2,\ 0$. The rank-one choice loses only the second mode.

Why an outside competitor cannot do better

For isometric outer factors, compression $C(B) = Q_L^* B Q_R$ cannot increase rank or either matrix norm. Every ambient rank-k competitor therefore supplies an admissible core competitor. The lifted core residual has exactly the core residual norm. These two facts transfer a core optimum to the entire ambient matrix space, not just to competitors already lying in the supplied ranges.

For a d-dimensional diagonal core and $0 \le k < d$, the spectral lower bound has a concrete witness. A rank-at-most-k competitor has a nonzero kernel vector in the first $k+1$ coordinates. On that vector the diagonal matrix acts with norm at least σ_{k+1} times the vector norm. The competitor cannot cancel that direction.

The Frobenius proof uses an orthonormal basis of the competitor's kernel. Its row weights lie in $[0, 1]$ and sum to the kernel dimension. Weighting the decreasing squared singular values forces at least the discarded tail. The two norm arguments are distinct; one is not a graph of the other.

> **Read this as:** Optimality is not inferred from the appearance of the singular-value plot. It follows from compression and a lower bound valid against every rank-bounded competitor.

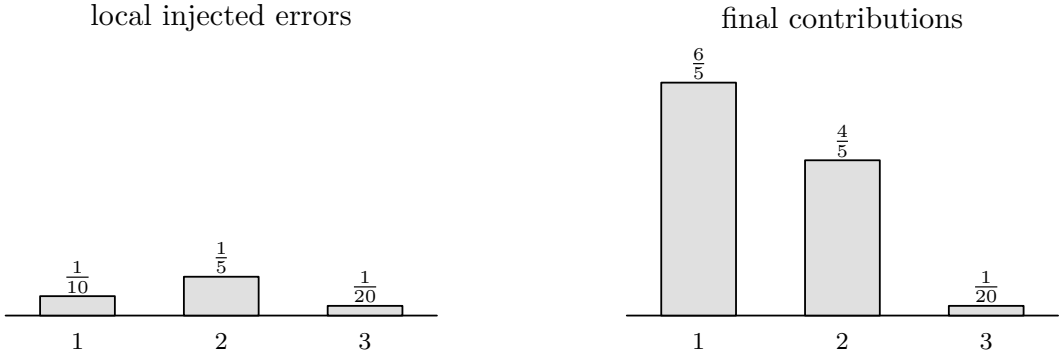
FACTOR PERTURBATION VERSUS RECOMPRESSION

Errors enter at different places

Main text: Theorems 5.8–5.9, p. 17.

Let the exact scalar factors be $(2, 3, 4)$ and start from $p_0 = 1$. If additive errors $(1/10, 1/5, 1/20)$ enter *after* each partial multiplication, then

$$\widehat{p}_j = a_j \widehat{p}_{j-1} + e_j, \qquad \widehat{p}_3 - p_3 = 12e_1 + 4e_2 + e_3 = \frac{41}{20}.$$



$\widehat{p}_j = a_j \widehat{p}_{j-1} + e_j$, with $(a_1, a_2, a_3) = (2, 3, 4)$.

The same local numbers, a different model

If instead the factors themselves are perturbed, the computed product is $(4 + e_3)(3 + e_2)(2 + e_1)$. Its error is $402/125$, not $41/20$. The factor-perturbation telescoping formula contains already-perturbed later factors; the additive recompression formula contains the original later factors.

The visual comparison uses the additive model. The first error is multiplied by both remaining factors. The last error is not amplified at all. Summing only the local error magnitudes gives $7/20$, which is smaller than the actual final error and therefore not a valid bound here.

In a matrix chain

The corresponding formulas replace scalar magnitudes with operator norms and respect multiplication order. A bound can be conservative when contributions cancel, but it must not omit possible amplification. Singular-tail errors from a recompression step become useful only after they are inserted into the correct global error model.

> **Read this as:** An error tolerance needs an entry point. A bound for perturbing factors and a bound for compressing partial products are not interchangeable.

SUPPORT IS AN INCLUSION

Two routes can cancel exactly

Main text: Theorems 2.5 and 6.1; Corollary 6.3, pp. 5 and 18.

Let K have transfers $0 \to 1$, $0 \to 2$, $1 \to 3$, and $2 \to 3$, with coefficients $1, 1, 1, -1$. There are two length-two routes from packet zero to packet three, but their contributions sum to zero.

K

0	0	0	0
1	0	0	0
1	0	0	0
0	1	-1	0

path counts at length two

0	0	0	0
0	0	0	0
0	0	0	0
2	0	0	0

$$1 + (-1) = 0$$

$$K^2 = 0$$

Structural support is an upper bound. It does not encode destructive cancellation.

$$(K^2)_{30} = K_{31}K_{10} + K_{32}K_{20} = 1 - 1 = 0, \qquad K^2 = 0.$$

The matrix of path counts sees the two possibilities and forgets the signs. It therefore predicts possible support but not actual support. Replacing the inclusion $\operatorname{supp}(AB) \subseteq \operatorname{supp}(A) \circ \operatorname{supp}(B)$ by equality would be wrong even in this four-dimensional model.

What an acyclic graph does guarantee

If the longest strict directed path has h edges, then no term of K^{h+1} can survive. Thus $K^{h+1} = 0$. This is an upper bound on the nilpotence index, not an assertion that $K^h \neq 0$. Here the graph has paths of length two, while coefficient cancellation already gives $K^2 = 0$.

A structurally permitted block may also vanish because a product of rectangular maps is zero before any sum is taken. A support graph cannot distinguish that mechanism from cancellation among separate routes. The actual block coefficients are therefore part of a proof-bearing operator representation.

> **Read this as:** Use graph reachability to exclude impossible transfers. Use the algebra to determine which permitted transfers actually survive.

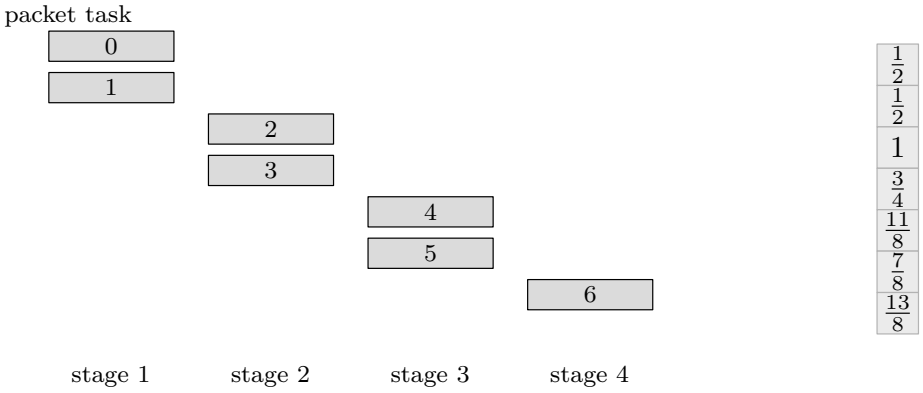
TOPOLOGICAL LEVELS WITH EXACT OUTPUTS

A solve has a dependency clock

Main text: Theorem 6.5, Corollary 6.6, and Theorem 10.3, pp. 18–19 and 32.

For seven scalar packets, put two on the diagonal and -1 on each declared incoming edge. The edges are $0 \to 2$, $1 \to 2$, $1 \to 3$, $2 \to 4$, $3 \to 4$, $3 \to 5$, $4 \to 6$, and $5 \to 6$. Solve $(2I + K)x = \mathbf{1}$.

$$x = (2I + K)^{-1}\mathbf{1}$$



Read the stages as dependencies

Packets zero and one are ready first. Packets two and three use those values. Packets four and five then become available, followed by packet six. For instance,

$$x_4 = \frac{1 + x_2 + x_3}{2} = \frac{11}{8}, \qquad x_6 = \frac{1 + x_4 + x_5}{2} = \frac{13}{8}.$$

All entries in the displayed solution are checked by multiplying the original triangular matrix by the resulting vector.

When is four a minimum?

There are four vertices on a longest dependency path. Under the paper's unit-duration, unlimited-concurrency model, four stages are both necessary and attainable. If only one execution slot is available, seven tasks require seven slots of work. If task durations differ, the relevant critical path is weighted.

The theorem is not a universal GPU scheduling result. Transfers, synchronization, contention, and nonunit task costs change the model. The exact benefit here is that independent rows in one level need not wait for one another's mathematical results.

> **Read this as:** A topological schedule separates dependencies from resources. Its optimal depth is conditional on the task-duration and concurrency model that defines a stage.

SCHUR FILL AND ITS CANCELLATION

Elimination creates new interactions

Main text: Theorem 6.7 and Proposition 6.8, pp. 19–20.

Retain the first two coordinates of

$$A = \begin{pmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}.$$

The third coordinate is hidden. Its invertible block is [2], so eliminating it produces

$$S = A_{RR} - A_{RH}A_{HH}^{-1}A_{HR} = \begin{pmatrix} 3/2 & -1/2 \\ -1/2 & 3/2 \end{pmatrix}.$$

A before elimination	$S = A_{RR} - A_{RH}A_{HH}^{-1}A_{HR}$	cancelling variant
2 0 1 0 2 1 1 1 2	$\frac{3}{2}$ $-\frac{1}{2}$ $-\frac{1}{2}$ $\frac{3}{2}$	$\frac{3}{2}$ 0 0 $\frac{3}{2}$

A hidden route creates $-1/2$; adding $+1/2$ in the original block cancels it.

The hidden route has a sign

The retained coordinates had no direct coupling, yet both touch the hidden coordinate. The resulting correction couples them by $-1/2$. This is fill: a new active retained block produced by an eliminated route.

Now change both original retained off-diagonal entries to $+1/2$. The same hidden correction cancels them, leaving the diagonal matrix $\frac{3}{2}I$. Thus a structural route can create or remove visible coupling depending on the direct coefficient already present.

A stronger inverse statement

For an invertible finite packet matrix, inverse support lies in the directed transitive closure. A reachable-coordinate subspace is invariant under the matrix. Injectivity of its restriction implies surjectivity in finite dimension, so the inverse preserves it as well. This proof uses finite-dimensional invertibility, not a convergent geometric series.

The checker verifies both Schur complements and, in each invertible example, equality between S^{-1} and the retained block of A^{-1}.

> **Read this as:** Removing coordinates does not remove their influence. It changes the retained operator, with possible fill, cancellation, and new conditioning constraints.

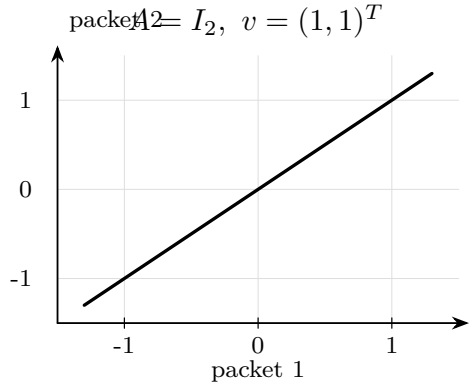
A STRICT KRYLOV INCLUSION

Invariant does not mean independently addressable

Main text: Proposition 7.2 and Theorem 7.3, p. 21.

Take $A = I_2$, split into the two coordinate packets, and start with $v = (1, 1)^T$. Every packet is invariant. Nevertheless every polynomial applies the *same* scalar to both coordinates:

$$p(A)v = p(1)(1, 1)^T, \qquad \dim \mathcal{K}_m(A, v) = 1 \quad (m \geq 1).$$



$$p(A)v = p(1)(1, 1)^T$$

$$\dim \mathcal{K}_m(A, v) = 1$$

Independent local spans have dimension 2.

The direct sum of the two local Krylov spans is all of $\mathbb{Q}^2$. It permits independent coefficients in the two packets. The global Krylov space does not. This is why the manuscript states an inclusion, not an unconditional equality.

What breaks the linkage?

With $A = \mathrm{diag}(1, 2)$, the vectors v and $Av = (1, 2)^T$ are independent. Two polynomial samples now distinguish the packet labels. Alternatively, allowing the packet projectors as extra address operators separates the coordinates even when $A = I_2$. Both changes enlarge the addressing power, but in different ways.

When invariance is only approximate

If $A = D + K$ with D packet diagonal, then

$$\|A^j - D^j\| \leq jM^{j-1}\|K\|, \qquad M = \max\{\|A\|, \|D\|\}.$$

This controls departure from the decoupled iterate. It is not a claim that the dimension or numerical conditioning of a Krylov basis varies continuously. Near dependence can remain difficult even when the operator error is small.

> **Read this as:** A single global polynomial can lock otherwise invariant packets together. Independence of subspaces is not the same as independence of the allowed actions.

BLOCK JACOBI AS A GEOMETRIC CHANGE

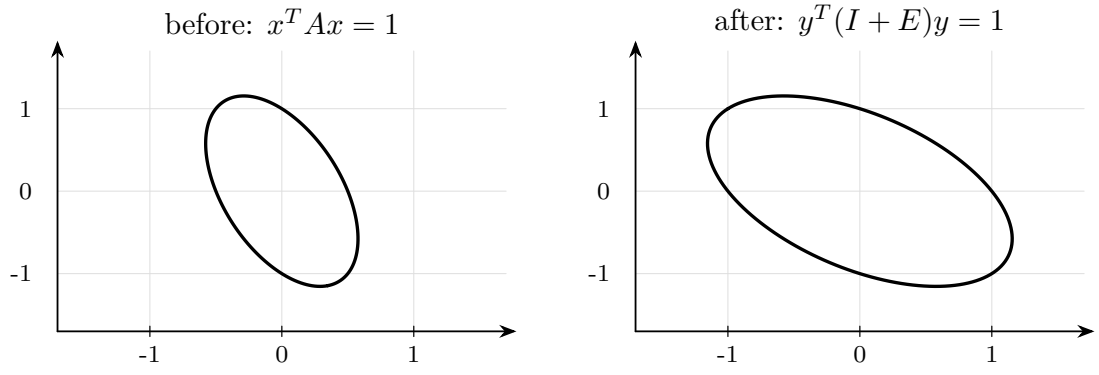
Measure coupling after normalization

Main text: Theorem 7.4 and Corollary 7.6, p. 22.

Let

$$A = \begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix}, \quad D = \operatorname{diag}(4,1), \quad B = D^{-1/2} = \operatorname{diag}(1/2,1).$$

Then $BAB = I + E = \begin{pmatrix} 1 & 1/2 \\ 1/2 & 1 \end{pmatrix}$. Its exact eigenvalues are $1/2$ and $3/2$, giving condition number three.



Exact normalized eigenvalues: $1/2, 3/2$. Hence $\kappa_2 = 3$.

The unit circles of the energy norms

The left ellipse is $x^T Ax = 1$. Writing $x = By$ gives the right ellipse $y^T(BAB)y = 1$. The drawing samples these explicitly defined curves; the norm and eigenvalue equalities are checked symbolically.

Here $\|E\|_2 = 1/2$. The manuscript's general inequality

$$\kappa_2(D^{-1/2}AD^{-1/2}) \le \frac{1+\eta}{1-\eta}, \qquad \|E\|_2 \le \eta < 1,$$

is attained. The raw off-diagonal entry is one; its meaningful size is one-half only after normalization by the diagonal energy scales.

Keep the convergence input separate

The condition bound can be inserted into a conjugate-gradient convergence estimate under that method's positive-definite assumptions. The CG estimate is one of the separately tracked classical inputs. The algebraic normalization theorem does not certify every stopping rule, floating-point iteration, or arbitrary preconditioner.

> **Read this as:** A small raw coupling is not scale-invariant. Normalize against the diagonal blocks before deciding whether the coupling is weak enough for a condition bound.

DISCOVERY AND PERTURBATION CERTIFICATES

Twelve small blocks, one coherent error

Main text: Theorem 8.1 and Proposition 9.2, pp. 23 and 26.

Split $\mathbb{Q}^4$ into scalar packets and let

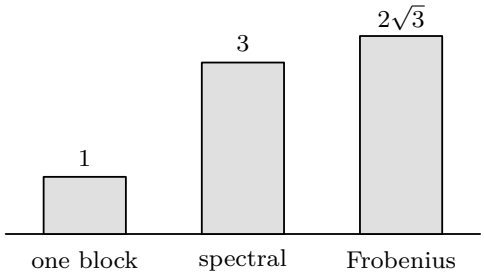
$$A = I + E, \qquad E = \frac{1}{100}(\mathbf{1}\mathbf{1}^T - I).$$

Dropping every off-diagonal block leaves I. Each discarded entry is only $1/100$, but the total error is not bounded by that single-entry size.

$100E$: twelve omitted entries

0	1	1	1
1	0	1	1
1	1	0	1
1	1	1	0

error norms, in units of $1/100$



$\|E\|_F^2 = 12/10^4$, while $E\mathbf{1} = (3/100)\mathbf{1}$.

Two norms answer different questions

There are twelve omitted entries, so $\|E\|_F^2 = 12/10^4$. The all-one vector is an eigenvector with eigenvalue $3/100$, while the three orthogonal directions have eigenvalue $-1/100$. Hence $\|E\|_2 = 3/100$.

For orthogonal coordinate blocks, the squared Frobenius norms add exactly. Operator norms instead require a suitable bound or further spectral information. The maximum omitted block norm is generally insufficient.

Input sensitivity is additional information

The packet perturbation estimate is

$$\|(\widehat{A} - A)x\| \le \sum_e \|E_e\| \, \|P_{\alpha(e)}x\|.$$

It can be much tighter for an input that scarcely occupies an affected source packet. To obtain a uniform operator bound, the input dependence must be bounded uniformly over unit vectors. A certificate for one input is not automatically a certificate for every right-hand side.

These are a posteriori checks for supplied projectors. They do not discover the best packet system or prove that a thresholding heuristic is globally optimal.

> **Read this as:** A proposed compression needs a residual certificate in the requested norm and input scope. Small individual blocks can align into a larger collective error.

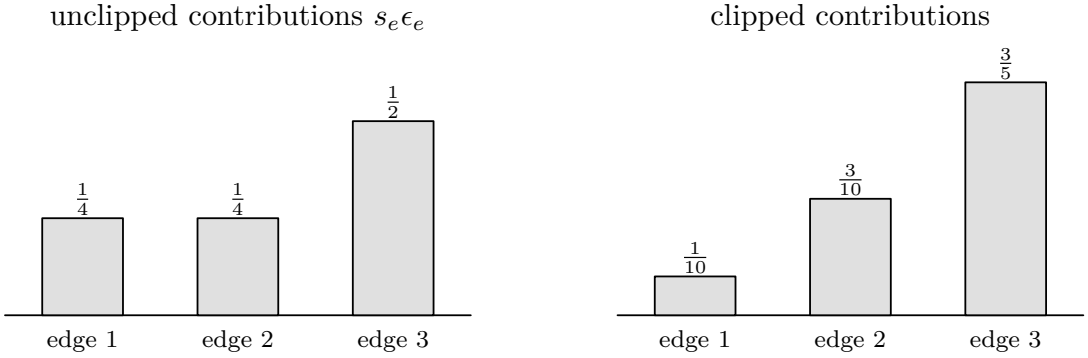
CONTINUOUS AND CLIPPED PRECISION

Spend error where it is cheapest

Main text: Theorems 8.4–8.5, p. 24.

Take sensitivities $s = (1, 2, 4)$, cost coefficients $c = (1, 1, 2)$, and budget $\tau = 1$. Minimizing $\sum_e c_e \log(1/\epsilon_e)$ subject to $\sum_e s_e \epsilon_e = 1$ gives

$$\epsilon_e = \frac{c_e}{4 s_e}, \qquad \epsilon = (1/4, 1/8, 1/8).$$



Both allocations spend exactly one unit of error budget.

The budget is weighted

The three contributions to the error allowance are $(1/4, 1/4, 1/2)$. Equal cost coefficients give equal weighted contributions, not equal tolerances. The third edge has twice the cost coefficient of the second, which offsets its doubled sensitivity in this example.

Now impose upper tolerances $m = (1/10, 1/2, 1/2)$. The first edge must stop at its tighter cap. With multiplier $\lambda = 10/3$, the clipped solution is

$$\epsilon_e = \min\{m_e, c_e/(\lambda s_e)\}, \qquad \epsilon = (1/10, 3/20, 3/20).$$

Its weighted contributions are $(1/10, 3/10, 3/5)$ and still sum to one. The checker verifies the active cap, the multiplier equations, and the budget exactly.

From tolerance to a real data type

The continuous objective is a model. Actual kernels offer discrete precisions and may use different accumulation types. A tolerance below one does not identify a particular hardware format. The finite option program on the next page is the correct next step once admissible costs and error bounds have been supplied.

> **Read this as:** Precision allocation is an optimization over certified sensitivities and costs. It does not infer error bounds from a data-type name alone.

THREE EDGES, TWENTY-SEVEN ASSIGNMENTS

An exact budget is not a greedy choice

Main text: Theorem 8.7 and Remark 8.8, p. 26.

Each edge has three allowed (integer weight, cost) options:

$$1 : (0, 7), (2, 3), (4, 1),$$
$$2 : (0, 6), (1, 4), (3, 1),$$
$$3 : (0, 9), (2, 5), (5, 1).$$

There are exactly $3^3 = 27$ full assignments. The table stores the minimum cost at each *exact* weight; columns zero through eight are displayed.

	0	1	2	3	4	5	6	7	8
edges 0	0	∞	∞	∞	∞	∞	∞	∞	∞
edges 1	7	∞	3	∞	1	∞	∞	∞	∞
edges 2	13	11	9	7	7	4	∞	2	∞
edges 3	22	20	18	16	14	12	12	9	8

Columns are exact weights; the budget optimum is the minimum across allowed columns.

Read the final row correctly

At budget four, minimize across columns zero through four. The optimum costs 14: choose weights $(2, 0, 2)$ with costs $(3, 6, 5)$. The zero-weight choice on edge two is its first, more expensive option. A locally cheap choice can consume too much of the shared budget.

The checker compares the recurrence with complete enumeration for every budget from zero through twelve. Infeasible states remain $+\infty$; they are not zero-cost states.

Quantization must be conservative

Suppose real errors are $3/50$ and $2/25$, and one integer unit represents $1/10$. Rounding down gives two zero weights, even though the true sum is $7/50 > 1/10$. Rounding up gives a certified upper envelope instead. The budget method is exact for its integers; it cannot repair unsafe integers supplied by a preprocessing step.

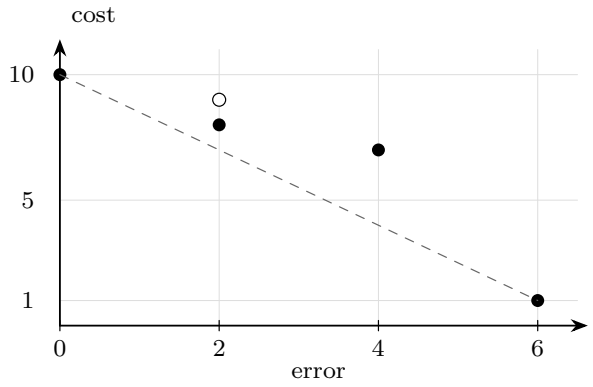
> **Read this as:** The table optimizes the declared option set exactly. Accuracy depends separately on the validity and conservativeness of the option weights.

FIVE PRUNINGS, ONE NONCONVEX FRONTIER

A penalty can miss the budget optimum

Main text: Theorem 9.4, Corollary 9.7, and Theorem 9.8, pp. 27–30.

A root r has children a, b, each with two terminal children. Keeping r costs one with error six; splitting it adds overhead one. Keeping either a or b costs three with error two. Splitting a costs four with zero error, and splitting b costs five with zero error.



Budget $B = 2$: cost 8

Penalty optimum: root or full split

$8 + 2\lambda \le 1 + 6\lambda$
and $8 + 2\lambda \le 10$
cannot both hold.

The five complete prunings have (cost, error) pairs

$$(1, 6),\ (7, 4),\ (9, 2),\ (8, 2),\ (10, 0).$$

At budget two the unique cheapest pruning is $\{a_0, a_1, b\}$, costing eight. It is Pareto optimal, but never minimizes cost $+\lambda$ err for any $\lambda \ge 0$.

See the incompatible inequalities

To beat the root, that pruning would require $8 + 2\lambda \le 1 + 6\lambda$, hence $\lambda \ge 7/4$. To beat the full split, it would require $8 + 2\lambda \le 10$, hence $\lambda \le 1$. No multiplier meets both conditions.

The exact budget recurrence retains the point because it enumerates admissible weight allocations. A sweep of Lagrangian parameters sees only supported frontier points. The two compiler formulations are therefore genuinely different, even on this tiny tree.

> **Read this as:** A penalty parameter is not a universal substitute for an explicit budget. Discrete, nonconvex frontiers can contain useful optima that no penalty selects.

LOCAL DEPENDENCIES AND STRICT MARGINS

Recompile the changed part, not everything

Main text: Theorems 9.11–9.12, pp. 30–31.

Use the same tree with $\lambda = 2$. Increase only the leaf cost at a_0 from two to three. The exact subtree values change at a_0, its parent a, and the root r. Every value in the b subtree remains untouched.

$a0$	$a1$	a	$b0$	$b1$	b	r
2	2	4	2	3	5	10
3	2	5	2	3	5	11

Top: old values. Bottom: new values. Only a0, a, and r change.

Why the update is local

A dynamic-programming value depends only on the local keep/split data and its children's values. A changed leaf can therefore affect its ancestors but not a disjoint sibling subtree. Recomputing the dependency closure yields the same global optimum as a fresh traversal. In this example, the root value increases from ten to eleven.

Strict gaps protect decisions

Before the update, the root keep score is 13 and its split score is 10, giving gap three. If each score changes by less than 3/2, their strict ordering is retained. Allowing exactly 3/2 in both directions can produce a tie, so the strict inequality in the stability statement matters.

The bound applies to the completed scores after child subproblems have been evaluated. Bounds on raw leaf perturbations must first be propagated to those scores. Shared caches, overlapping subproblems, or global rank constraints can introduce dependencies outside a tree and invalidate this simple locality argument.

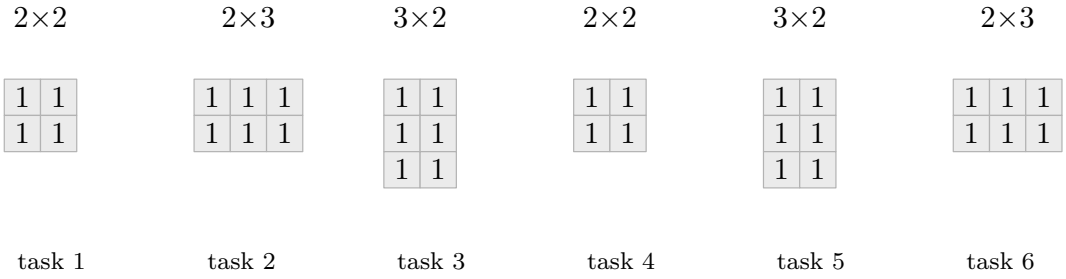
> **Read this as:** Local recompilation is exact because the model has local dependencies. The locality is a mathematical hypothesis, not an automatic property of every compiler optimization.

A LAUNCH MODEL IS NOT A BANDWIDTH MODEL

Batch shapes, account for traffic

Main text: Proposition 3.6, Theorem 3.8, and Proposition 10.2, pp. 9 and 31.

Six independent blocks have the displayed dimensions. When right-hand-side count, scalar type, accumulation type, and kernel contract also agree within each class, they can be grouped into three batches. That reduces the modeled launch count from six to three without changing their arithmetic.



Three shape classes, six independent tasks.

Grouping does not remove arithmetic, data transfer, or shared-output dependencies.

Independence must be real

The batching statement assumes compatible backend support and independent block tasks. If several blocks accumulate into one target, the reduction or accumulation policy is additional work. Equal matrix dimensions alone do not remove that dependency. Different accumulation types also define different shape classes under the paper's convention.

Count the data separately

For three diagonal 2×2 blocks, eight-byte scalars, and q right-hand sides, the stated one-pass model moves

$$B_{\text{stream}} = 8 \cdot 3(4 + 2q + 4q) = 96 + 144q \text{ bytes.}$$

It gives 240 bytes at $q = 1$ and 672 bytes at $q = 4$. It deliberately assumes no reuse across edges. A cache-aware implementation may have another traffic model, but it must state one.

Given operation capacity P and bandwidth W, $T \geq \max(F/P, B/W)$ is a lower bound, not an achieved runtime. Launch overhead, imperfect overlap, and utilization can make the actual time much larger.

> **Read this as:** Grouping can remove launches while leaving the same products and data movement. Arithmetic, traffic, and scheduling are related constraints, not one interchangeable cost.

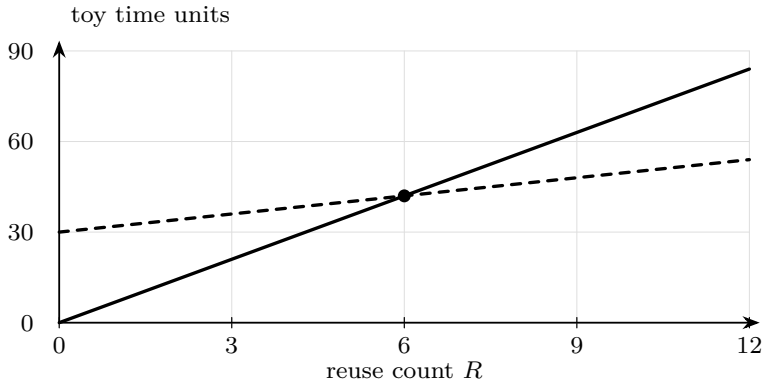
SETUP AND REUSE IN ONE LINEAR MODEL

The first win comes after the tie

Main text: Theorem 10.4 and Corollaries 10.5–10.6, pp. 32–33.

Suppose the baseline costs seven units per use. A candidate costs 30 units to set up and two per use. These are synthetic constants chosen to expose the exact threshold:

$$T_b(R) = 7R, \qquad T_c(R) = 30 + 2R.$$



solid: $7R$

dashed: $30 + 2R$

tie at 6
first win at 7

At $R = 6$, both cost 42. The candidate becomes strictly faster at $R = 7$, where it costs 44 instead of 49. The minimum profitable integer reuse count is

$$\left\lfloor \frac{30}{7 - 2} \right\rfloor + 1 = 7.$$

Replacing the floor-plus-one rule with a ceiling would be wrong when the ratio is already an integer and strict improvement is required.

Which costs belong to setup?

Discovery, compilation, host/device transfer, graph capture, and factor preparation can have different reuse lifetimes. If a matrix changes but its shape does not, a captured schedule might remain reusable while its numerical factors do not. A single setup number is meaningful only after those lifetimes are fixed.

Materialization has the opposite tradeoff: pay once for an explicit representation, then use a potentially faster dense downstream operation. It pays off only when the explicit per-use cost is lower and the reuse count exceeds the corresponding conversion threshold.

> **Read this as:** A faster steady-state kernel can lose end-to-end. State the reuse regime and count the tie separately from the first strict win.

A BENCHMARK ROW IS A CONTRACT

Same output before comparing times

Main text: Proposition 2.9 and Definitions 11.1–11.7, pp. 6 and 33–34.

Consider four synthetic timing rows for a product. Row B returns factors, while row A returns an explicit matrix. A ratio $12/2 = 6$ does not compare two implementations of the same returned object. Adding the decoding cost changes the candidate to row C.

row	output	scope	time
A	explicit matrix	materialization included	12
B	low-rank factors	factor product only	2
C	explicit matrix	factor product + decode	14
D	explicit matrix	different input precision	1

Synthetic times: A versus C is contract-aligned. A versus B is not a sixfold speedup.

What a fair match retains

The input values, dimensions, layout, source precision, correctness reference, returned output, and timing scope must agree. Row D changes the input precision; that can be a useful accuracy tradeoff, but it is not the same case merely because the output has the same shape.

If a downstream task consumes factors directly, row B has a valid compressed-output contract. It should be compared with another method that supplies that downstream result, not forced to materialize an unused array. Fairness does not mean every algorithm must use the same internal representation.

How the original measurements are treated here

The preserved manuscript contains its historical CPU/GPU corpus. Replaying its figure generator redraws those saved observations. It does not rerun the recorded device, establish present hardware performance, or transform logical avoided work into physically executed floating-point operations. No new GPU timing is added by this appendix.

Exact representation gains, device gains, kernel gains, and amortization gains can be isolated only through appropriately matched rows. A total ratio that mixes them is still an observation, but not a decomposition theorem.

> **Read this as:** A timing number without its case and scope is incomplete data. Preserve the mathematical output contract before claiming a speedup.

FACTORS, APPLICATION, AND AN INDEPENDENT REFERENCE

Close the loop on the running example

Main text: Theorems 2.3, 3.4, and 5.1, pp. 4, 7, and 13.

Return to the two operators from V−9. The exact product representation has diagonal part $D_A D_B$ and two-column correction factors U_C, V_C. Apply it to the original $x = (1, -1, 2, 0, -2, 3)^T$.

$$V_C^T x = (2, 13)^T, \qquad (AB)x = D_A D_B x + U_C(V_C^T x).$$

$V_C^T x$	$D_A D_B x$	$U_C(V_C^T x)$	$(AB)x$
2	1	25	26
13	−5	21	16
	18	25	43
	6	21	27
	−12	25	13
	44	21	65

Two intermediate scalars and six output coordinates, with an exact dense reference.

A complete reader calculation

The local contribution is $(1, -5, 18, 6, -12, 44)^T$. The correction is $(25, 21, 25, 21, 25, 21)^T$. Their sum is

$$(26, 16, 43, 27, 13, 65)^T.$$

The checker forms the dense matrices A and B independently, multiplies them, applies the product to x, and compares all six components. The dense array is a validation reference, not a requirement of the factored application route.

Choose what happens next

For another input, reuse the factors. For a required explicit matrix, pay materialization. For a long product chain, track rank growth and decide whether recompression is useful. If approximation is introduced, transport its error through the later operators rather than reporting only a local singular tail.

This small example does not predict a speedup: at dimension six, implementation overhead can dominate. Its purpose is to make the semantics auditable. The same identity can then be used inside a larger, separately benchmarked workload.

> **Read this as:** The full route is: represent the exact operator, retain every interaction, return the requested object, and validate against an independent reference. Performance is a further experiment.

FORMAL, EXACT, NUMERICAL, AND EMPIRICAL EVIDENCE

What this edition certifies

Reading source: sixth-block supplement, pp. 1 and 7–8; current release ledger.

The preserved v0.6.0 release records 56 of 56 manuscript-proved finite items, 392 named theorem declarations in 40 modules, and 66 separately counted construction audits. Of the theorem declarations, 19 are reused foundation results. These are inherited records, not a new Lean run for this editorial edition.

Evidence	Its precise use here
Saved Lean records	Preserve the exact hypotheses and declared finite formalization boundary.
New exact checks	Reproduce the appendix's matrices, identities, integer models, and finite optima. They are not extracted Lean proofs.
Vector drawings	Display explicit arrays, schedules, and sampled geometric formulas. Pixel accuracy is not algebraic certification.
Replayed Python tests	Exercise the preserved software on its stated finite test cases. Floating-point tests retain numerical tolerances.
Historical GPU corpus	Remains a record of the supplied original cases and environment. It is not a new measurement.

The recompression distinction

The sixth block derives best-rank and singular-tail conclusions from ordered thin-SVD equations, rather than assuming an optimality certificate. It does not independently establish general SVD existence, certify a floating-point QR/SVD algorithm, or identify every result with a canonical singular-values API. Those are separate statements.

The four classical entries remain visible

The manuscript separately tracks Eckart–Young–Mirsky, the conjugate-gradient energy estimate, the floating-point product model, and simultaneous spectral decomposition. The strengthened ordered-SVD proof does not silently promote all associated existence and numerical-algorithm claims.

Limits of exact compilation

The optimizer needs a finite candidate tree with additive data. Nonlocal cache effects, overlapping decompositions, global rank coupling, and unrestricted discovery can destroy its optimal substructure. No new generic dense-matrix exponent or universal accelerator claim follows from this appendix.

> **Read this as:** This is a reader-facing reconstruction of the paper's finite concepts, with reproducible examples and preserved source evidence. It changes exposition, not the formal theorem contract.